



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Geoinformatyka**

Specjalność:

Michał Jan Wilkos

Nr albumu: 307370

Projekt i implementacja oprogramowania do analizy przestrzenno-czasowej danych treningowych z urządzeń GPS

Design and implementation of software for spatial-temporal
analysis of training data from GPS devices

Praca magisterska
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki
pod kierunkiem dr Michała Klisowskiego

Lublin 2026

Spis treści

Wstęp	5
1 Podstawy teoretyczne optymalizacji kodu i przetwarzania danych przestrzennych	7
1.1 Architektura komputerowa a efektywność obliczeniowa	7
1.1.1 Organizacja i hierarchia pamięci podręcznej procesora	7
1.1.2 Równoległe wykonywanie operacji	9
1.1.3 Wektoryzacja instrukcji procesora	9
1.2 Systemy pozycjonowania satelitarnego	11
1.2.1 Zasada działania pozycjonowania satelitarnego	11
1.2.2 Formaty zapisu danych telemetrycznych	12
2 Przetwarzanie o wysokiej wydajności w języku Python	17
2.1 High-Performance Computing	17
2.1.1 Języki stosowane w HPC	19
2.1.2 Wbudowane struktury danych języka Python a wydajność obliczeniowa	19
2.1.3 NumPy	21
2.2 Pomocnicze biblioteki	24
3 Szczegóły implementacji	25
3.1 Struktura projektu	25
3.2 Moduł przetwarzania danych TCX	26
3.3 Moduł przetwarzania danych treningowych	29
3.3.1 Obliczanie dystansu	29
3.3.2 Obliczanie przewyższeń	31
3.3.3 Analiza parametru tętna serca	31
3.3.4 Analiza wydolności biomechanicznej i tlenowej	34
3.3.5 Segmentacja aktywności podczas treningu	38

4	Przeprowadzenie analizy porównawczej	45
4.1	Specyfikacja sprzętowa	45
4.2	Przygotowanie danych i implementacja narzędzi badawczych	46
4.2.1	Generowanie danych testowych	46
4.2.2	Program do analizy czasu wykonania algorytmów analitycznych .	49
4.3	Analiza czasu wykonania algorytmów analitycznych	51
4.3.1	Czas przetwarzania	51
4.3.2	Analiza współczynnika przyspieszenia	52
4.3.3	Szczegółowa analiza profilu operacji analitycznych	53
4.4	Analiza wykorzystania pamięci podręcznej	55
4.4.1	Metodyka i problematyka profilowania sprzętowego	55
4.4.2	Analiza chybień pamięci podręcznej	57
4.4.3	Podsumowanie analizy wydajnościowej i sprzętowej	59
	Podsumowanie	61
	Bibliografia	63
	Spis listingów	65
	Spis rysunków	67
	Spis tabel	69

Wstęp

W czasach rosnącej świadomości na temat zdrowego trybu życia oraz dynamicznego rozwoju technologii pomiaru sesji treningowych coraz więcej osób decyduje się na monitorowanie swoich aktywności fizycznych, a popularność zegarków sportowych i urządzeń z modułami GPS rośnie z roku na rok. Jednak generowanie ogromnych ilości danych o przebytych dystansach i parametrach treningowych niesie ze sobą problemy nie tylko w kontekście interpretacji, ale przede wszystkim obliczeń, szczególnie w przypadku przetwarzania informacji przestrzenno-czasowych.

W procesie tworzenia oprogramowania analitycznego ważną rolę odgrywa wybór odpowiedniej metody przetwarzania dużych zbiorów danych. Chociaż język Python oferuje mnogość narzędzi, wiele rozwiązań różni się pod względem efektywności czasowej i pamięciowej. Często stosowane standardowe struktury danych, takie jak listy, mogą okazać się niewystarczające przy skomplikowanych i obciążających pamięć obliczeniach. Te zagadnienia oraz chęć sprawdzenia różnic w wydajności stanowiły inspirację do stworzenia aplikacji, która posłuży jako środowisko testowe dla tych metod.

Praca będzie się koncentrować na opracowaniu oraz zaimplementowaniu oprogramowania do analizy przestrzenno-czasowej danych treningowych, z uwzględnieniem porównania efektywności różnych metod algorytmów analitycznych. Głównym założeniem jest nie tylko stworzenie funkcjonalnego narzędzia, ale także zbadanie i zestawienie wydajności operacji wykonywanych na podstawowych strukturach danych w porównaniu do rozwiązań wektorowych.

Struktura pracy zostanie podzielona na cztery rozdziały. Pierwszy z nich omówi podstawy teoretyczne optymalizacji kodu i przetwarzania danych przestrzennych, ze wskazaniem na architekturę pamięci podręcznej oraz systemy pozycjonowania satelitarne. Drugi rozdział będzie przedstawiać charakterystykę przetwarzania o wysokiej wydajności w języku Python oraz architekturę biblioteki NumPy. W trzecim rozdziale przedstawione zostaną szczegóły implementacyjne, w tym organizacja kodu, struktura projektu oraz moduły odpowiedzialne za przetwarzanie danych treningowych. Ostatni rozdział zostanie poświęcony przeprowadzeniu analizy porównawczej wydajności zastosowanych rozwiązań w odniesieniu do czasu przetwarzania algorytmów analitycznych oraz wykorzystania pamięci podręcznej procesora.

Rozdział 1

Podstawy teoretyczne optymalizacji kodu i przetwarzania danych przestrzennych

Ten rozdział poruszy tematy dotyczące pamięci, podstaw przetwarzania danych, które stanowią bazę do zrozumienia mechanizmów optymalizacji oprogramowania, a także przybliży architektury systemów nawigacji satelitarnej oraz różne formaty przechowujące dane przestrzenne.

1.1 Architektura komputerowa a efektywność obliczeniowa

1.1.1 Organizacja i hierarchia pamięci podręcznej procesora

Współczesne procesory wykonują obliczenia szybciej, niż system pamięci jest w stanie dostarczyć im dane. Jest to problem znany jako „wąskie gardło” występujący w architekturze von Neumanna, według której dane są przechowywane w pamięci wraz z instrukcjami kodu programu[1]. Problem ten określa się również jako ścianę pamięci (ang. *memory wall*). To sprawia, że w aplikacjach przetwarzających duże ilości danych problemem staje się opóźnienie w dostępie do pamięci RAM, a nie moc obliczeniowa rdzenia.

Aby złagodzić ten problem, architektury komputerowe wykorzystują wielopoziomową pamięć podręczną, która stanowi swoisty bufor pomiędzy procesorem a pamięcią RAM. Dzieli się ona na trzy poziomy, z czego pierwszy z nich jest najszybszy, a trzeci – najwolniejszy[1]. Jest to mniejsza pamięć, ale znacznie szybsza od pamięci operacyjnej komputera. W celu uzyskania wysokiej wydajności często zarządza się danymi tak, aby procesor jak najczęściej znajdował potrzebne informacje w pamięci podręcznej, ograniczając kosztowne pobrania z pamięci głównej RAM.

Fundamentem wydajnego wykorzystania pamięci podręcznej jest zasada lokalności odwołań:

- Lokalność czasowa: Dane, do których uzyskano dostęp, prawdopodobnie będą potrzebne ponownie w niedługim czasie.
- Lokalność przestrzenna: Jeśli program odwołuje się do określonego adresu pamięci, z dużym prawdopodobieństwem wkrótce odwoła się do sąsiednich adresów[1].

Z perspektywy programisty i wyboru struktur danych w projekcie, lokalność przestrzenna ma bardzo ważne znaczenie. Procesory nie pobierają z pamięci pojedynczych bajtów, lecz całe linie pamięci podręcznej. Struktury takie jak tablice, gdzie elementy są ułożone w pamięci w kolejności jeden za drugim, są pod tym względem bardzo wydajne. Pobranie pierwszego elementu automatycznie ładuje do cache'u kolejne, co pozwala na płynne dostarczanie danych[2].

Struktury oparte na węzłach i wskaźnikach, takie jak listy połączone lub drzewa charakteryzują się rozrzuceniem elementów po sterpie pamięci. Przejście do kolejnego elementu wymaga skoku do losowego adresu, co często skutkuje chybieniem pamięci podręcznej i wstrzymaniem pracy procesora na czas pobrania danych z RAM[2].

Współczesne procesory wielordzeniowe zazwyczaj implementują trójpoziomowy system pamięci podręcznej. Pamięć podręczna pierwszego poziomu jest najszybszą i najmniejszą spośród wszystkich trzech poziomów. Dzieli się na dwie niezależne sekcje – L1i dla instrukcji oraz L1d dla danych. Taki podział zapobiega konfliktom między pobieraniem kodu a odczytem danych operacyjnych. Rozmiar tej pamięci waha się od 32 do 48 KB na rdzeń. [1]

Pamięć drugiego poziomu stanowi bufor między pamięcią L1 a wolniejszymi zasobami. Jej pojemność waha się od 256 KB do 2 MB na rdzeń. W przypadku „chybienia” w L1, procesor szuka danych w L2, co zapobiega wstrzymaniu pracy.[1]

Pamięć trzeciego poziomu jest największą pamięcią podręczną w procesorze. Zazwyczaj jest ona współdzielona przez wszystkie rdzenie w procesorze. Jej rozmiar może wynosić od kilkunastu do ponad stu megabajtów. Głównym zadaniem pamięci podręcznej trzeciego poziomu jest synchronizacja danych między rdzeniami, a także redukcja odwołań do pamięci RAM.[1]

Ważną kwestią do poruszenia w tym temacie jest mechanika transferu danych. System pamięci nie przesyła pojedynczych bajtów czy zmiennych. Komunikacja między pamięciami RAM, L3, L2 i L1 odbywa się za pomocą tzw. linii pamięci podręcznej (ang. *cache lines*). Oznacza to, że pojedyncze odwołanie do pamięci przez procesor powoduje pobranie całego 64-bajtowego bloku.[1]

1.1.2 Równoległe wykonywanie operacji

Przez dekady wzrost wydajności aplikacji opierał się na zwiększaniu częstotliwości taktowania procesorów, lecz ograniczenia architektoniczne sprawiły, że dalsze zwiększanie prędkości pojedynczego rdzenia stało się nieefektywne. Zamiast tego, zaczęto zwiększać liczbę rdzeni w procesorach, co doprowadziło do zmiany w projektowaniu systemów i aplikacji – przyspieszania pojedynczych instrukcji w stronę równoległych obliczeń[3].

Do klasyfikacji architektur równoległych stosuje się taksonomię zaproponowaną przez Michaela Flynna w 1966 roku

- SISD (ang. *Single Instruction, Single Data*): Architektura von Neumanna, gdzie jeden procesor wykonuje jedną instrukcję na jednej danej. Jest to model sekwencyjny.
- SIMD (ang. *Single Instruction, Multiple Data*): Jedna instrukcja sterująca wykonuje tę samą operację na wielu elementach danych jednocześnie. Jest to podstawa wektoryzacji.
- MISD (ang. *Multiple Instruction, Single Data*): Wiele instrukcji operuje na jednej danej.
- MIMD (ang. *Multiple Instruction, Multiple Data*): Wiele procesorów lub rdzeni wykonuje niezależnie różne instrukcje na różnych danych. Jest to podstawowy model równoległości[3].

Architektura MIMD jest realizowana w obecnych komputerach za pomocą procesorów wielordzeniowych. W kontekście analizy danych treningowych pozwala to na podział pracy na mniejsze, niezależne zadania. System może analizować kilka niezależnych plików jednocześnie, gdzie każdy plik obsługiwany jest przez osobny wątek procesora. W związku z tym system może działać w przykładowy sposób:

- Rdzeń 1 analizuje trasę biegu z poniedziałku.
- Rdzeń 2 analizuje trasę rowerową z wtorku.
- Rdzeń 3 analizuje spacer ze środy.

Ponieważ wynik analizy jednego działania nie zależy od wyniku drugiego, rdzenie te nie muszą się ze sobą komunikować ani na siebie czekać.[3]

1.1.3 Wektoryzacja instrukcji procesora

Według taksonomii zaproponowanej przez Michaela J. Flynna, tradycyjne jednostki obliczeniowe operują w modelu SISD. Oznacza to, że pojedyncza instrukcja maszynowa modyfikuje stan pojedynczego parametru w jednym cyklu maszynowym. Alternatywą, która zdominowała współczesne jednostki CPU, jest model SIMD. Paradygmat ten

eksploatuje zjawisko równoległości na poziomie danych (ang. *Data-Level Parallelism*), umożliwiając zaaplikowanie tej samej operacji (np. dodawania, mnożenia) do wektora elementów rozlokowanych w dedykowanym, szerokim rejestrze sprzętowym.[3]

Zysk wydajnościowy płynący z wektoryzacji można opisać za pomocą prawa Amdahla. Jeżeli P oznacza ułamek programu podatny na zrównoleglenie wektorowe, a N oznacza liczbę elementów przetwarzanych jednocześnie w jednym rejestrze wektorowym, to teoretyczne maksymalne przyspieszenie S (ang. *speedup*) określa wzór:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

Z powyższego równania wynika, że skuteczność technologii SIMD jest ściśle zdeterminowana przez to, jak duża część algorytmu daje się wyrazić w postaci niezależnych operacji wektorowych.[3]

Sama translacja instrukcji na postać wektorową nie gwarantuje proporcjonalnego przyspieszenia. Optymalizacja SIMD napotyka na dwa fundamentalne wyzwania, które muszą zostać uwzględnione w projektowaniu algorytmów. Najczęstszym powodem takiej nieefektywności jest niewystarczająca przepustowość szyny pamięci pomiędzy RAM a pamięcią podręczną procesora. Procesor jest w stanie wykonywać operacje SIMD znacznie szybciej, niż system pamięci dostarcza nowe dane.[3]

Bezpośrednią konsekwencją specyfiki ładowania danych wektorowych jest konieczność zmiany systemów projektowania struktur danych w pamięci (ang. *Data-Oriented Design*). Klasyczne programowanie obiektowe naturalnie układa dane w sposób AoS, czyli tablice struktur (ang. *Array of Structures*). Przylegające do siebie w pamięci bajty reprezentują różne atrybuty pojedynczego obiektu (np. x, y, z, kolor, czas). Utrudnia to załadowanie np. wyłącznie współrzędnych x z kolejnych elementów.

$$\text{AoS} = \left[\underbrace{(\text{Poz}_1, \text{Prę}_1)}_{\text{Obiekt}_1}, \underbrace{(\text{Poz}_2, \text{Prę}_2)}_{\text{Obiekt}_2}, \dots, \underbrace{(\text{Poz}_N, \text{Prę}_N)}_{\text{Obiekt}_N} \right] \quad (1.1)$$

Dlatego też aplikacje zorientowane na wysoką wydajność obliczeniową korzystają z ułożenia SoA, czyli struktury tablic (ang. *Structure of Arrays*), w której atrybuty tego samego typu tworzą oddzielne, ciągłe obszary pamięci.[4]

$$\text{SoA} = \begin{cases} \text{Tablica pozycji} = [\text{Poz}_1, \text{Poz}_2, \dots, \text{Poz}_N] \\ \text{Tablica prędkości} = [\text{Prę}_1, \text{Prę}_2, \dots, \text{Prę}_N] \end{cases} \quad (1.2)$$

Jeżeli algorytm odwołuje się do pamięci w sposób losowy lub nieliniowy, czyli tak jak ma to miejsce w przypadku układu tablic struktur, wczytana 64-bajtowa linia będzie zawierać wiele bezużytecznych w danym momencie informacji. To sprawia, że procesor szybko wykorzysta całą pojemność pamięci podręcznej pierwszego poziomu.

Z drugiej strony, gdy dane są ułożone w pamięci sekwencyjnie (układ struktur tablic), pobranie pierwszego elementu spowoduje załadowanie kolejnych elementów do pamięci L1. Dzięki temu rozwiązaniu procesor, wykonując instrukcje wektorowe, znajdzie kolejne potrzebne dane już w najszybszej pamięci.[4]

Aby zilustrować, w jaki sposób procesor i jego pamięć podręczna widzą układ tych danych podczas operacji wymagającej odczytu wyłącznie jednego atrybutu (np. samej pozycji, z całkowitym pominięciem prędkości), można zestawzić ze sobą oba podejścia. W przypadku tradycyjnego modelu AoS, wskaźnik przechodzący przez pamięć musi załadować pełen obiekt, odczytać pozycję, a następnie przeskoczyć niepotrzebną w danej chwili prędkość. Generuje to narzut i zanieczyszcza linię pamięci podręcznej:

$$\text{Odczyt AoS: } \left[\underbrace{\mathbf{Poz}_1}_{\text{odczyt}}, \underbrace{\mathbf{Prę}_1}_{\text{omijane}}, \underbrace{\mathbf{Poz}_2}_{\text{odczyt}}, \underbrace{\mathbf{Prę}_2}_{\text{omijane}}, \dots, \underbrace{\mathbf{Poz}_N}_{\text{odczyt}}, \underbrace{\mathbf{Prę}_N}_{\text{omijane}} \right] \quad (1.3)$$

Z kolei w architekturze SoA odczytywane są wyłącznie pożądane informacje. Procesor sprawnie pobiera do rejestrów spójny, ciągły wektor jednorodnych danych, podczas gdy niezwiązane z daną operacją tablice pozostają całkowicie nietknięte i nie obciążają przepustowości pamięci:

$$\text{Odczyt SoA: } \left\{ \begin{array}{l} \left[\mathbf{Poz}_1, \mathbf{Poz}_2, \dots, \mathbf{Poz}_N \right] \\ \text{ciągły odczyt (tylko potrzebne dane)} \\ \left[\mathbf{Prę}_1, \mathbf{Prę}_2, \dots, \mathbf{Prę}_N \right] \\ \text{całkowicie nietknięte bloki pamięci} \end{array} \right. \quad (1.4)$$

1.2 Systemy pozycjonowania satelitarnego

Global Positioning System (GPS), oficjalnie znany jako NAVSTAR GPS, to satelitarny system nawigacyjny stworzony i zarządzany przez Departament Obrony Stanów Zjednoczonych. Należy on do szerszej rodziny systemów klasy GNSS (ang. *Global Navigation Satellite System*), do której zaliczają się również europejski system Galileo, rosyjski GLONASS oraz chiński BeiDou. Choć w mowie potocznej akronim „GPS” stosuje się uniwersalnie do określenia każdego rodzaju nawigacji satelitarnej, w dosłownym ujęciu odnosi się on do konkretnego amerykańskiego systemu satelitarnego.[5]

1.2.1 Zasada działania pozycjonowania satelitarnego

Zasada działania opiera się na pomiarze czasu propagacji sygnału. Jeżeli odbiornik wie, o której godzinie sygnał został wysłany przez satelitę i o której dotarł do anteny, może obliczyć czas przelotu (ang. *Time of Flight*). Odległość od satelity to iloczyn cza-

su przelotu i prędkości światła. Odległość ta determinuje miejsce w przestrzeni, gdzie znajduje się odbiornik.[5]

W ujęciu matematycznym, jeśli satelita emituje sygnał w czasie t^s , a odbiornik rejestruje go w czasie t^r , to teoretyczna odległość geometryczna d jest wyrażana wzorem:

$$d = c \cdot (t^r - t^s) \quad (1.5)$$

gdzie c oznacza prędkość światła. W praktyce jednak metoda ta wymaga uwzględnienia dodatkowych czynników. Wyznaczenie odległości od jednego satelity lokalizuje odbiornik na powierzchni sfery. Zastosowanie dwóch satelitów zawęży pozycję do okręgu stanowiącego przecięcie dwóch sfer, a z trzech – do dwóch punktów w przestrzeni.

Aby jednoznacznie określić swoją pozycję w trójwymiarowym układzie współrzędnych (np. WGS 84), odbiornik musi zatem wyznaczyć cztery niewiadome: współrzędne przestrzenne x, y, z oraz błąd zegara δt . Wymaga to jednoczesnego odbioru sygnału z minimum czterech satelitów, co pozwala na ułożenie i rozwiązanie układu równań składającego się z czterech równań nieliniowych w postaci:[5]

$$rho = \sqrt{(x_1 - x)^2 + (y_1 - y)^2 + (z_1 - z)^2} + c \cdot \delta t \quad (1.6)$$

1.2.2 Formaty zapisu danych telemetrycznych

Wraz z popularyzacją systemów informacji geograficznej (GIS) i osobistych urządzeń nawigacyjnych, zaistniała potrzeba ustandaryzowania zapisu w sposób umożliwiający łatwą wymianę informacji między różnymi platformami. Doprowadziło to do powstania formatów opartych na języku znaczników XML (ang. *Extensible Markup Language*), z których najpowszechniejszym stał się GPX (ang. *GPS Exchange Format*).

GPX zdefiniował podstawowy system opisu tras (ang. *Routes*), śladów (ang. *Tracks*) i punktów (*Waypoints*). Jednakże, wraz z rozwojem telemetry sportowej i smartwatchy, nawigacyjny charakter formatu GPX okazał się niewystarczający, co wymusiło stworzenie nowych formatów, integrujących dane przestrzenne z fizjologicznymi.[6]

Ewolucja ta poszła w dwóch głównych kierunkach. Z jednej strony, na potrzeby urządzeń sportowych opracowano wydajny, binarny format FIT (ang. *Flexible and Interoperable Data Transfer*). Dzięki swojej skompresowanej strukturze pozwala on na bardzo oszczędny zapis danych lokalizacyjnych oraz parametrów biomechanicznych i fizjologicznych (takich jak moc czy tętno) bezpośrednio w pamięci zegarków i liczników rowerowych.[7] Z drugiej strony, naturalnym krokiem było rozwinięcie dotychczasowej technologii XML w stronę sportu. Tak powstał format TCX (ang. *Training Center XML*), który stał się bezpośrednim łącznikiem między czysto geograficznym GPX-em a nowoczesną telemetryą treningową.

Format TCX (Training Center XML) został zaprojektowany i wprowadzony przez firmę Garmin w 2007 roku jako część oprogramowania Garmin Training Center. W przeciwieństwie do uniwersalnego formatu GPX, TCX został zbudowany od podstaw z myślą o analizie sportowej, rekreacji i monitorowaniu wysiłku fizycznego.

Z punktu widzenia inżynierii danych, plik TCX jest dokumentem XML podlegającym walidacji przez schemat XSD (XML Schema Definition), opublikowany w przestrzeni nazw <http://www.garmin.com/xmlschemas/TrainingCenterDatabase/v2>. [8]

Listing 1.1: Przykładowy plik TCX

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <TrainingCenterDatabase
3   xsi:schemaLocation="http://www.garmin.com/xmlschemas/
   TrainingCenterDatabase/v2 http://www.garmin.com/xmlschemas/
   TrainingCenterDatabasev2.xsd"
4   xmlns:ns5="http://www.garmin.com/xmlschemas/ActivityGoals/v1"
5   xmlns:ns3="http://www.garmin.com/xmlschemas/ActivityExtension/
   v2"
6   xmlns:ns2="http://www.garmin.com/xmlschemas/UserProfile/v2"
7   xmlns="http://www.garmin.com/xmlschemas/TrainingCenterDatabase
   /v2"
8   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:ns4="http://www.garmin.com/xmlschemas/
   ProfileExtension/v1">
9   <Activities>
10    <Activity Sport="Running">
11      <Id>2025-05-06T04:58:19.000Z</Id>
12      <Lap StartTime="2025-05-06T04:58:19.000Z">
13        <TotalTimeSeconds>335.479</TotalTimeSeconds>
14        <DistanceMeters>1000.0</DistanceMeters>
15        <MaximumSpeed>3.4149999618530273</MaximumSpeed>
16        <Calories>58</Calories>
17        <AverageHeartRateBpm>
18          <Value>146</Value>
19        </AverageHeartRateBpm>
20        <MaximumHeartRateBpm>
21          <Value>158</Value>
22        </MaximumHeartRateBpm>
23        <Intensity>Active</Intensity>
24        <TriggerMethod>Manual</TriggerMethod>

```

```

25     <Track>
26         <Trackpoint>
27             <Time>2025-05-06T04:58:19.000Z</Time>
28             <Position>
29                 <LatitudeDegrees>51.23510423116386</
                    LatitudeDegrees>
30                 <LongitudeDegrees>22.55628313869238</
                    LongitudeDegrees>
31             </Position>
32             <AltitudeMeters>171.39999389648438</AltitudeMeters>
33             <DistanceMeters>0.8999999761581421</DistanceMeters>
34             <HeartRateBpm>
35                 <Value>113</Value>
36             </HeartRateBpm>
37             <Extensions>
38                 <ns3:TPX>
39                     <ns3:Speed>0.8679999709129333</ns3:Speed>
40                     <ns3:RunCadence>0</ns3:RunCadence>
41                 </ns3:TPX>
42             </Extensions>
43         </Trackpoint>
44         <Trackpoint>
45     </Track>
46 </Lap>
47 </Activity>
48 </Activities>
49 </TrainingCenterDatabase>

```

Główną cechą odróżniającą TCX od innych formatów jest jego orientacja na „aktywność” (ang. *Activity*), a nie tylko i wyłącznie na „ścieżkę” (ang. *Track*). Plik TCX modeluje zbiór danych telemetrycznych w oparciu o zagnieżdżoną hierarchię:

- TrainingCenterDatabase: Główny węzeł dokumentu, który zawiera w sobie aktywności użytkownika.
- Activities / Activity: Reprezentuje pojedynczy trening. Atrybut Sport węzła Activity natychmiast kategoryzuje rodzaj ruchu (np. Running, Biking).
- Lap (Okrążenie): Jednostka podziału treningu w TCX. TCX dzieli aktywność na logiczne okrążenia (definiowane ręcznie przez użytkownika za pomocą przycisku na

zegarku lub automatycznie, np. co 1 kilometr). Węzeł Lap przechowuje od razu zagregowane statystyki: TotalTimeSeconds (całkowity czas), DistanceMeters (dystans), MaximumSpeed (prędkość maksymalna) oraz Calories (spalone kalorie) itp.

- Track / Trackpoint: Wewnątrz każdego okrążenia znajduje się ścieżka, stanowiąca zbiór punktów pomiarowych (Trackpoint).[8]

Pojedynczy punkt pomiarowy w formacie TCX stanowi połączenie danych z różnych sensorów. Podczas gdy tradycyjny odbiornik GPS dostarcza głównie współrzędnych i czasu, zegarek sportowy synchronizuje w tym samym momencie dane z pulsometru, barometru i akcelerometru.

Struktura informacyjna węzła Trackpoint obejmuje:

- <Time>: Znacznik czasu w formacie ISO 8601 (np. 2023-05-14T08:30:00.000Z).
- <Position>: Blok przestrzenny zawierający <LatitudeDegrees> i <LongitudeDegrees>. Reprezentacja bazuje na układzie WGS 84 w formie stopni dziesiętnych. Element ten jest opcjonalny – np. w przypadku biegu na bieżni mechanicznej, zegarek generuje punkty TCX pozbawione bloku <Position>, ale zawierające tętno i czas.
- <AltitudeMeters>: Wysokość, najczęściej z dokładnego czujnika barometrycznego kalibrowanego sygnałem GPS.
- <DistanceMeters>: Skumulowany dystans od początku okrążenia.
- <HeartRateBpm>: Informacja o tętnie w uderzeniach na minutę.[8]

Z uwagi na fakt, że oryginalny schemat TCX powstał kilkanaście lat temu, nie przewidywał on nowoczesnych metryk, takich jak moc generowana podczas biegu, balans stóp czy saturacja krwi. Aby uniknąć konieczności ciągłego modyfikowania głównego schematu, Garmin wprowadził mechanizm rozszerzeń w oparciu o nowe przestrzenie nazw (ActivityExtension/v2). Wewnątrz węzła Trackpoint można umieścić blok <Extensions>, który pozwala na dołączanie dowolnych nowych danych.[8]

Rozdział 2

Przetwarzanie o wysokiej wydajności w języku Python

Ten rozdział poruszy tematy dotyczące architektury systemów High-Performance Computing (HPC) i specyfiki języka Python w celu zrozumienia jego ograniczeń wydajnościowych. Przybliżone zostaną również metody integracji standardowej implementacji z wydajnymi bibliotekami opartymi na niskopoziomowych rozwiązaniach.

2.1 High-Performance Computing

Przetwarzanie o wysokiej wydajności (ang. *High-Performance Computing*) to dziedzina informatyki zajmująca się agregacją mocy obliczeniowej w celu rozwiązywania złożonych problemów naukowych i inżynierskich, których realizacja na zwykłych komputerach byłaby niemożliwa ze względu na ograniczenia czasowe lub pamięciowe.

Systemy HPC, potocznie nazywane superkomputerami, pozwalają na wykonywanie bilionów, a nawet trylionów obliczeń w ciągu sekundy. Podstawową miarą wydajności w systemach HPC jest wskaźnik FLOPS (ang. *Floating-Point Operations Per Second*), określający liczbę operacji zmiennoprzecinkowych wykonywanych w ciągu jednej sekundy. Współczesne superkomputery osiągają wydajność rzędu petaFLOPS (10^{15} operacji na sekundę), a najpotężniejsze jednostki na świecie przekroczyły już barierę exaFLOPS (10^{18} operacji na sekundę).[9]

Współczesne systemy HPC rzadko są pojedynczymi, monolitycznymi maszynami. Opierają się na architekturze klastrowej, w której setki lub tysiące niezależnych serwerów jest połączonych w jedną, zintegrowaną strukturę. Podstawowe elementy sprzętowe klastra HPC obejmują:

- Węzły obliczeniowe (ang. *Compute Nodes*): Odpowiedzialne za wykonywanie właściwych obliczeń. Każdy węzeł to w praktyce autonomiczny serwer wyposażony we

własne procesory wielordzeniowe, pamięć operacyjną oraz coraz częściej w akceleratorach obliczeniowych, takie jak procesory graficzne.[10]

- Węzły dostępowe (ang. *Login Nodes*): Są to punkty wejściowe do systemu. Użytkownik loguje się do węzła dostępowego, aby kompilować kod, zarządzać swoimi danymi oraz zlecać zadania obliczeniowe. Węzły te nie służą do wykonywania ciężkich obliczeń, aby nie blokować dostępu innym użytkownikom.[10]
- Sieć połączeń: Ponieważ zadania w HPC są dzielone na wiele węzłów, które muszą nieustannie wymieniać między sobą dane, standardowe sieci Ethernet mogą okazać się niewystarczające. W systemach HPC stosuje się dedykowane sieci charakteryzujące się bardzo wysoką przepustowością i niskimi opóźnieniami. Dominującą technologią w tym obszarze jest InfiniBand.[3]
- Systemy pamięci masowej: Obliczenia HPC generują i konsumują terabajty danych. Zwykłe dyski twarde nie są w stanie poradzić sobie z takim wyzwaniem. Stosuje się więc systemy plików (np. Lustre), które pozwalają tysiącom procesów na jednoczesny odczyt i zapis danych, rozpraszając fizyczny zapis na setki dysków i serwerów plikowych.[10]

Aby w pełni wykorzystać potencjał klastra HPC, oprogramowanie musi zostać zaprojektowane tak, by działało równolegle. Wyróżnia się trzy główne paradygmaty programowania w takich środowiskach:

- Pamięć współdzielona (ang. *Shared Memory*): Wykorzystywana w obrębie pojedynczego węzła. Wiele wątków tego samego programu wykonuje się na różnych rdzeniach procesora, ale wszystkie mają dostęp do tej samej fizycznej przestrzeni pamięci. Standardem programistycznym dla tego modelu jest OpenMP.[3]
- Pamięć rozproszona (ang. *Distributed Memory*): Konieczna, gdy program wykonuje się na wielu różnych węzłach, z których każdy posiada własną, niezależną pamięć. Węzły muszą „rozmawiać” ze sobą, przesyłając komunikaty przez sieć połączeń. De facto standardem w tej dziedzinie jest biblioteka MPI (ang. *Message Passing Interface*).[3]
- Przetwarzanie heterogeniczne (ang. *Heterogeneous Computing*): Gdy obliczenia zlecano są do kart graficznych, stosuje się specyficzne języki i biblioteki, takie jak CUDA (dla środowiska NVIDIA) lub otwarty standard OpenCL.[3]

W środowisku HPC kod źródłowy to jedynie połowa sukcesu. Równie istotny jest sam proces tłumaczenia go na język maszynowy. Ośrodki superkomputerowe rzadko opierają się wyłącznie na standardowych kompilatorach typu open-source, często inwestując w kompilatory dedykowane konkretnej architekturze sprzętowej.

Kompilatory stworzone dla takich firm jak Intel, NVIDIA, czy AMD są w stanie wygenerować zestaw instrukcji dopasowany do mikroarchitektury procesora, na którym kod będzie uruchamiany.

2.1.1 Języki stosowane w HPC

W systemach HPC priorytetem jest absolutna maksymalizacja wydajności oraz minimalizacja czasu wykonania i zużycia pamięci. Z tego powodu dominującą rolę odgrywają języki kompilowane (tzw. języki niskiego poziomu), które oferują programiście bezpośrednią kontrolę nad zarządzaniem pamięcią operacyjną oraz architekturą sprzętową, a jednocześnie pozwalają kompilatorom na przeprowadzanie znacznej optymalizacji kodu wynikowego.

Język C jest powszechnie stosowany ze względu na wysoką wydajność, zwieżłość składni i minimalny narzut abstrakcji (ang. *overhead*). Operowanie na poziomie wskaźników pozwala na ścisłą integrację kodu z architekturą sprzętową. Ułatwia to optymalizację algorytmów oraz pozwala kompilatorom na skuteczną wektoryzację pętli. Ponadto język C pełni rolę uniwersalnego standardu w tworzeniu interfejsów binarnych, co umożliwia bezproblemową integrację oprogramowania i bibliotek tworzonych w innych technologiach.

Z kolei język C++ rozwija te możliwości, łącząc bazową wydajność języka C z paradigmatami programowania obiektowego. W kontekście HPC istotne znaczenie ma zastosowanie szablonów, które pozwalają na przeniesienie części logiki decyzyjnej z czasu wykonania programu na czas jego kompilacji. Umożliwia to projektowanie elastycznych struktur danych bez dodatkowego obciążenia wydajnościowego. Współczesne standardy języka C++ dostarczają również natywne mechanizmy współbieżności, wspierając tym samym rozwój zaawansowanych aplikacji wielowątkowych.[11]

2.1.2 Wbudowane struktury danych języka Python a wydajność obliczeniowa

Język Python, dzięki swojej elastyczności, dynamicznemu typowaniu oraz niezwykle czytelnej składni, stał się jednym z najpopularniejszych języków programowania na świecie. Jednak mechanizmy, które czynią go tak przystępnym w procesie tworzenia oprogramowania, stanowią jednocześnie jego największą barierę w kontekście przetwarzania o wysokiej wydajności. Standardowe implementacje opierają się na architekturze, która przedkłada uniwersalność nad optymalizację sprzętową.

W językach niskiego poziomu, takich jak C czy C++, standardowa tablica jest ciągłym blokiem pamięci przechowującym wartości jednego, z góry określonego typu (np. 64-bitowe liczby całkowite), co zapewnia lokalność przestrzenną oraz odgórnie znany rozmiar. Wbudowana lista w języku Python działa na zupełnie innej zasadzie. Pod wzglę-

dem implementacyjnym nie jest to tablica przechowująca bezpośrednio dane, lecz dynamiczna tablica wskaźników. Oznacza to, że każdy element na liście jest jedynie adresem lub inaczej odniesieniem do rzeczywistego obiektu umieszczonego w pamięci operacyjnej komputera.[12]

Dodatkowo, z uwagi na dynamiczne typowanie i fakt, że w Pythonie wszystko jest obiektem, nawet prosta liczba całkowita nie jest tylko ciągiem bitów. W implementacji opartej na listach Pythona, każda liczba jest strukturą `PyObject`, która oprócz samej wartości zawiera minimum dwa dodatkowe pola:

- Licznik referencji (ang. *reference count*) – przechowujący informację o liczbie zmiennych wskazujących na dany obiekt.
- Wskaźnik na typ obiektu – pozwalający interpreterowi określić w czasie wykonywania, z jakim typem danych ma do czynienia.[13]

Taka architektura generuje spory narzut pamięciowy (ang. *memory overhead*). Podczas gdy pojedyncza, 64-bitowa liczba całkowita w języku C zajmuje dokładnie 8 bajtów, ten sam obiekt w Pythonie może zajmować znacznie więcej bajtów w zależności od potrzeby, a do tego dochodzą jeszcze 8-bajtowe wskaźniki w samej liście. W przypadku przetwarzania milionów punktów z danych GPS, ilość alokowanej pamięci drastycznie rośnie, obciążając przepustowość szyny pamięci.

Jeszcze poważniejszą konsekwencją jest wpływ list na pamięć podręczną procesora. Jak wspomniano w rozdziale poświęconym hierarchii pamięci, procesory osiągają maksymalną wydajność, gdy dane są ułożone sekwencyjnie. Przetwarzanie natywnej listy w Pythonie zmusza procesor do wykonywania skoków. Odczyt elementu wymaga najpierw pobrania wskaźnika, a następnie wyciągnięcia informacji o nim, co prowadzi do odczytu danych z losowego, nieprzewidywalnego adresu w pamięci RAM.[4]

Prowadzi to do nieustannych "chybień" pamięci podręcznej (ang. *cache misses*). Linie pamięci cache są wypełniane bezużytecznymi danymi, a procesor spędza większość cykli zegara na bezczynnym oczekiwaniu na dostarczenie danych z wolnej pamięci głównej RAM. Architektura ta jest klasycznym przykładem skrajnie rozproszonego układu, przeciwnego do optymalnego wzorca Structure of Arrays (SoA).[2]

Z perspektywy wektoryzowania lub zrównoleglenia na poziomie danych, natywne listy Pythona są praktycznie nieużyteczne dla kompilatorów i procesorów wektorowych. Mechanizmy wektoryzacji wymagają, aby instrukcje były aplikowane do paczek jednorodnych, przylegających do siebie w pamięci danych [3].

Ponieważ lista w Pythonie może przechowywać elementy różnych typów w sposób dynamiczny (np. liczby całkowite, zmiennoprzecinkowe i ciągi znaków obok siebie), kompilator JIT lub sprzęt nie są w stanie z góry zagwarantować spójności typów w czasie

wykonania. Przed każdą operacją arytmetyczną interpreter musi przeprowadzić kosztowną procedurę dynamicznego sprawdzania typów (ang. *type checking*) dla każdego pojedynczego obiektu. To skutecznie blokuje możliwość wykorzystania szerokich rejestrów wektorowych współczesnych architektur HPC.[14]

W celu ominięcia tych barier architektonicznych, biblioteki analityczne zorientowane na wydajność zmuszone są do omijania wewnętrznych struktur Pythona i implementowania własnych, ciągłych bloków pamięci w niższych warstwach (np. poprzez język C), co stanowi fundament działania takich narzędzi jak NumPy.

2.1.3 NumPy

NumPy to jedna z najważniejszych i najbardziej fundamentalnych bibliotek języka Python, przeznaczona do zaawansowanych obliczeń matematycznych. Stanowi ona podstawę dla wielu innych popularnych narzędzi do analizy danych i uczenia maszynowego, takich jak Pandas, SciPy, Scikit-Learn czy TensorFlow. Głównym celem powstania pakietu NumPy było dostarczenie wydajnych struktur danych oraz zoptymalizowanych algorytmów do operacji na dużych zbiorach informacji, co w zwykłym języku Python jest procesem mało wydajnym ze względu na jego interpretowany charakter i dynamiczne typowanie.[15]

Tablice wielowymiarowe i podejście do typów danych

Sercem biblioteki jest obiekt `ndarray` (ang. *N-dimensional array*), który reprezentuje wielowymiarową tablicę elementów. W przeciwieństwie do implementacji opartej na standardowych strukturach języka Python, gdzie listy mają charakter dynamiczny i mogą przechowywać obiekty różnych typów jednocześnie, biblioteka NumPy wprowadza rygorystyczne podejście do zarządzania danymi. Tablica `ndarray` jest z definicji homogeniczna – wszystkie jej elementy muszą reprezentować ten sam typ, który jest określany przez atrybut `dtype`. [15]

NumPy oferuje rozbudowany zestaw zdefiniowanych typów numerycznych, opartych na standardach znanych z języka C (np. `int32`, `int64`, `float64`). Zdefiniowanie konkretnego typu pozwala na alokację całej tablicy w ciągłym bloku pamięci operacyjnej. Eliminuje to konieczność przechowywania metadanych dla każdego pojedynczego elementu, co ma miejsce w wbudowanych listach Pythona. Ta jednorodność i ciągłość pamięciowa skutkują znacznym zmniejszeniem narzutu pamięciowego oraz pozwalają na błyskawiczny dostęp do danych z poziomu pamięci podręcznej procesora.[15]

Wektoryzacja w obliczeniach numerycznych

Ważnym paradygmatem determinującym wysoką wydajność biblioteki NumPy jest wektoryzacja (ang. *vectorization*). Tradycyjne operacje na kolekcjach danych w zwykłym Pythonie wymagają stosowania jawnych pętli iteracyjnych (np. pętli `for`), co w przypadku wielowymiarowych zbiorów danych prowadzi do ogromnego spadku wydajności.

Wektoryzacja polega na zastąpieniu tych pętli operacjami aplikowanymi na całe kolekcje danych. Znaczna część biblioteki NumPy została zaimplementowana w kompilowanych, niskopoziomowych językach C oraz Fortran. Dzięki temu operacje matematyczne omijają narzut związany z interpretacją kodu w maszynie wirtualnej Pythona. Co więcej, algorytmy NumPy są zaprojektowane tak, aby wykorzystywać sprzętowe instrukcje wektorowe nowoczesnych procesorów. Pozwala to na wektoryzowanie i zrównoleglenie obliczeń na poziomie sprzętowym, np. poprzez jednoczesne dodawanie wielu elementów dwóch wektorów w jednym takcie procesora.[15]

Broadcasting

Niezwykle istotną cechą biblioteki NumPy jest wbudowany mechanizm transmitowania (ang. *broadcasting*). Umożliwia on wykonywanie operacji arytmetycznych na tablicach o różnych, ale kompatybilnych wymiarach.

W standardowej algebrze liniowej operacje na macierzach wymagają ścisłego dopasowania ich kształtów. NumPy potrafi zinterpretować intencję programisty i w przypadku braku dopasowania "rozszerza" mniejszą tablicę, aby jej wymiary odpowiadały większej. Dzieje się to bez fizycznego kopiowania danych w pamięci, co pozwala na oszczędność zasobów systemowych podczas złożonych operacji na tablicach.[15]

Podstawowe operacje matematyczne i statystyczne

Pakiet dostarcza szeroki wachlarz wbudowanych, zoptymalizowanych metod, do których należą między innymi:

- Algebra liniowa: operacje na wektorach i macierzach, obliczanie wyznaczników, odwracanie macierzy oraz rozwiązywanie układów równań liniowych.
- Funkcje uniwersalne (ufuncs): szybkie, zwektoryzowane funkcje matematyczne stosowane element po elemencie (np. funkcje trygonometryczne, logarytmiczne, wykładnicze).
- Statystyka: wydajne wyznaczanie miar tendencji centralnej i rozproszenia, takich jak średnia, mediana, odchylenie standardowe czy wariancja dla wielkich zbiorów danych.[15]

Porównanie operacji w Pythonie a NumPy

Aby zobrazować różnice pomiędzy klasycznym podejściem w Pythonie a wykorzystaniem biblioteki NumPy, posłużę do tego poniżej przedstawiony przykład operacji dodawania dwóch dużych wektorów liczb. Przykładowy kod realizuje ten sam cel w dwóch wariantach: z użyciem jawnej pętli w Pythonie oraz z wykorzystaniem wektoryzacji w NumPy. Porównanie to pozwala zaobserwować różnice zarówno w sposobie implementacji, jak i w podejściu do przetwarzania danych.

Listing 2.1: Porównanie operacji na listach Pythona a wektorach NumPy

```
1 import numpy as np
2
3 ROZMIAR = 10000000
4
5 # =====
6 # 1. Standardowa implementacja Python
7 # =====
8 lista_a = list(range(ROZMIAR))
9 lista_b = list(range(ROZMIAR))
10
11 wynik_python = []
12 for i in range(len(lista_a)):
13     wynik_python.append(lista_a[i] + lista_b[i])
14
15 # =====
16 # 2. Biblioteka NumPy
17 # =====
18 tablica_a = np.arange(ROZMIAR)
19 tablica_b = np.arange(ROZMIAR)
20
21 wynik_numpy = tablica_a + tablica_b
```

Powyższy listing 2.1 przedstawia różnicę w implementacji dodawania dwóch dużych zbiorów danych za pomocą standardowych mechanizmów języka Python oraz biblioteki NumPy. W pierwszej części (linie 6-13) operacja jest wykonywana przy użyciu tradycyjnej pętli `for`, która iteruje po każdym elemencie i dodaje go do nowej listy. W drugiej części (linie 16-21) zastosowano tablice NumPy. Dzięki mechanizmowi wektoryzacji dodawanie całych tablic sprowadza się do użycia operatora `+` bez konieczności jawnego pisania pętli. Kod z wykorzystaniem NumPy jest nie tylko krótszy i bardziej czytelny, ale przede wszystkim wykonuje się znacznie szybciej.

2.2 Pomocnicze biblioteki

Oprócz biblioteki NumPy, która stanowi fundament aplikacji, w projekcie wykorzystano również inne biblioteki pomocnicze. Pełnią one role na początkowych oraz końcowych etapach przetwarzania informacji – podczas pozyskiwania surowych danych oraz ich końcowej prezentacji. Są to biblioteki lxml oraz matplotlib.

Biblioteka lxml to jedno z najpopularniejszych narzędzi w języku Python przeznaczone do przetwarzania, analizy i modyfikacji dokumentów w formatach XML (ang. *Extensible Markup Language*) oraz HTML (ang. *Hypertext Markup Language*). Biblioteka oferuje mechanizmy parsowania iteracyjnego, co pozwala na przetwarzanie bardzo dużych plików „w locie”, bez konieczności ładowania całego drzewa dokumentu do pamięci komputera.[16]

Z kolei biblioteka matplotlib to popularna biblioteka w Pythonie służąca do tworzenia wizualizacji danych. Biblioteka matplotlib służy do przekształcania suchych danych liczbowych w czytelne grafiki. Za jej pomocą generowane są przejrzyste wykresy i diagramy.[17]

Rozdział 3

Szczegóły implementacji

Ten rozdział poruszy zagadnienia związane z praktyczną realizacją założeń projektowych. Omówiona zostanie ogólna struktura projektu oraz jego najważniejsze komponenty. Następnie szczegółowo przedstawione zostaną dwa podejścia do realizacji kodu: przy użyciu standardowych struktur Pythona oraz z wykorzystaniem biblioteki NumPy.

3.1 Struktura projektu

Projekt został podzielony na trzy główne katalogi. W folderze **data** przechowywane są surowe pliki wejściowe w formacie TCX, które stanowią materiał do analizy. Katalog **helping_scripts** zawiera narzędzia pomocnicze wykorzystywane podczas rozwoju projektu, w tym skrypt do generowania danych TCX (**generate.py**) oraz skrypty testujące wydajność (**np_test.py**, **py_test.py**). Z kolei główna logika znajduje się w folderze **scripts**. Tam znajdują się właściwe implementacje funkcji, które bazują na standardowych strukturach Pythona oraz tablicach NumPy, a także skrypt do parsowania plików TCX (**tcx_python.py**, **tcx_numpy.py**, **tcx_parser.py**). W tym folderze można znaleźć również skrypt mierzący wydajność czasową obu rozwiązań (**benchmark.py**) i moduł generujący wykresy z wynikami (**plot.py**).

- data
- helping_scripts:
 - generate.py
 - np_test.py
 - py_test.py
- scripts:
 - benchmark.py

- benchmark_functions.py
 - plot.py
 - tcx_numpy.py
 - tcx_parser.py
 - tcx_python.py
- results

3.2 Moduł przetwarzania danych TCX

Plik `tcx_parser.py` ma zaimplementowane dwie funkcje parsujące. Pierwsza z nich (listing 3.1) opiera się na wbudowanych strukturach języka Python, natomiast druga (listing 3.2) wykorzystuje mechanizmy biblioteki NumPy.

Listing 3.1: Funkcja parsująca

```

1 def parser_py(file):
2     ns = {
3         'tcx': 'http://www.garmin.com/xmlschemas/
           TrainingCenterDatabase/v2',
4         'ns3': 'http://www.garmin.com/xmlschemas/
           ActivityExtension/v2'
5     }
6     latitudes, longitudes, elevations, heart_rates, times,
       cadences = [], [], [], [], [], []
7     tree = etree.parse(file)
8
9     for tp in tree.iterfind('./tcx:Trackpoint', namespaces=ns):
10         pos = tp.find('tcx:Position', namespaces=ns)
11         if pos is not None:
12             lat_elem = pos.find('tcx:Position/tcx:
                LatitudeDegrees', namespaces=ns)
13             lon_elem = pos.find('tcx:Position/tcx:
                LongitudeDegrees', namespaces=ns)
14             if lat_elem is not None and lon_elem is not None:
15                 time_elem = tp.find('tcx:Time', namespaces=ns)
16                 if time_elem is not None and time_elem.text:
17                     time_str = time_elem.text.replace('Z', '
                        +00:00')
```

```

18         times.append(datetime.fromisoformat(time_str
19                               ).timestamp())
20     else:
21         times.append(0.0)
22         latitudes.append(float(lat_elem.text))
23         longitudes.append(float(lon_elem.text))
24         ele_elem = tp.find('tcx:AltitudeMeters',
25                             namespaces=ns)
26         elevations.append(float(ele_elem.text) if
27                             ele_elem is not None else 0.0)
28         hr_elem = tp.find('./tcx:HeartRateBpm/tcx:Value',
29                             namespaces=ns)
30         heart_rates.append(float(hr_elem.text) if
31                             hr_elem is not None else 0.0)
32         cad_elem = tp.find('./ns3:RunCadence',
33                             namespaces=ns)
34         cadences.append(float(cad_elem.text) if cad_elem
35                             is not None else 0.0)
36
37     return latitudes, longitudes, elevations, heart_rates, times
38         , cadences

```

Działanie funkcji rozpoczyna się od zdefiniowania słowników z przestrzeniami nazw (ang. *namespaces*), które są wymagane przez bibliotekę lxml do poprawnego nawigowania po drzewie plików Garmin TCX (listing 3.1, linie 2-5). Następnie deklarowanych jest sześć pustych list, które będą przechowywać informacje o zmiennych treningowych (listing 3.1, linia 6). W kolejnym kroku wykorzystana została metoda `iterfind` (listing 3.1, linia 9). Tworzy ona generator, który przetwarza węzły Trackpoint "w locie". Kod weryfikuje również, czy dany punkt posiada znacznik Position oraz współrzędne geograficzne (listing 3.1, linie 10-14). Jeśli ich brakuje, cały węzeł jest pomijany. Właściwe dane są dopisywane do list za pomocą metody `.append()` (listing 3.1, linie 18, 21, 24, 26, 28). W przypadku braku odczytu do listy dopisywana jest wartość domyślna 0.0 (listing 3.1, linia 28).

Listing 3.2: Funkcja parsująca (NumPy)

```

1 def parser_np(file):
2     ns = "{http://www.garmin.com/xmlschemas/
3         TrainingCenterDatabase/v2}"
4     ns3 = "{http://www.garmin.com/xmlschemas/ActivityExtension/
5         v2}"
6     tree = etree.parse(file)

```

```

5     trackpoints = tree.findall(f'./{ns}Trackpoint')
6     n = len(trackpoints)
7
8     latitudes = np.empty(n, dtype=np.float64)
9     longitudes = np.empty(n, dtype=np.float64)
10    elevations = np.empty(n, dtype=np.float64)
11    heart_rates = np.empty(n, dtype=np.float64)
12    times = np.empty(n, dtype=np.float64)
13    cadences = np.empty(n, dtype=np.float64)
14
15    i = 0
16    for tp in trackpoints:
17        pos = tp.find(f'{ns}Position')
18        if pos is not None:
19            lat_str = pos.findtext(f'{ns}LatitudeDegrees')
20            lon_str = pos.findtext(f'{ns}LongitudeDegrees')
21            if lat_str is not None and lon_str is not None:
22                time_str = tp.findtext(f'{ns}Time')
23                if time_str is not None:
24                    times[i] = datetime.fromisoformat(time_str.
25                                                         replace('Z', '+00:00')).timestamp()
26                else:
27                    times[i] = np.nan
28                latitudes[i] = float(lat_str)
29                longitudes[i] = float(lon_str)
30                ele_str = tp.findtext(f'{ns}AltitudeMeters')
31                elevations[i] = float(ele_str) if ele_str is not
32                    None else np.nan
33                hr_str = tp.findtext(f'./{ns}HeartRateBpm/{ns}
34                    Value')
35                heart_rates[i] = float(hr_str) if hr_str is not
36                    None else np.nan
37                cad_str = tp.findtext(f'./{ns3}RunCadence')
38                cadences[i] = float(cad_str) if cad_str is not
39                    None else np.nan
40            i += 1
41
42    return latitudes[:i], longitudes[:i], elevations[:i],

```

```
heart_rates[:i], times[:i], cadences[:i]
```

W przeciwieństwie do leniwej iteracji z poprzedniej wersji, tutaj funkcja `findall` od razu pobiera wszystkie węzły typu `Trackpoint` do pamięci (listing 3.2, linia 5). Pozwala to na określenie ich dokładnej liczby i przypisanie jej do zmiennej `n` (listing 3.2, linia 6). Znając liczbę punktów, tworzone są tablice przy pomocy funkcji `np.empty()` (listing 3.2, linie 8-13). Zmienna pomocnicza `i` pełni rolę licznika zapisanych punktów. Zamiast domyślnych wartości 0.0 dla brakujących atrybutów wpisywana jest wartość `np.nan` (listing 3.2, linie 22, 29, 31). Ostatecznie zwracane tablice są przycinane za pomocą składni `[:i]` (listing 3.2, linia 37), ponieważ ostateczna liczba zebranych punktów może być mniejsza niż zainicjalizowany na początku rozmiar tablic `n`.

3.3 Moduł przetwarzania danych treningowych

Pliki `tcx_python.py` oraz `tcx_numpy.py` zawierają główną logikę projektu. To tutaj znajdują się funkcje odpowiedzialne za główną logikę projektu oraz przetwarzanie danych TCX.

3.3.1 Obliczanie dystansu

Listing 3.3: Funkcje odpowiedzialne za obliczanie dystansu

```

1 def distance(lat1, lon1, lat2, lon2):
2     phi1, phi2 = math.radians(lat1), math.radians(lat2)
3     delta_phi = math.radians(lat2 - lat1)
4     delta_lambda = math.radians(lon2 - lon1)
5     a = math.sin(delta_phi / 2.0) ** 2 + math.cos(phi1) * math.
        cos(phi2) * math.sin(delta_lambda / 2.0) ** 2
6     c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
7     return R * c
8
9 def total_distance(latitudes, longitudes):
10     total_distance = 0.0
11     for i in range(1, len(latitudes)):
12         total_distance += distance(latitudes[i - 1], longitudes[
            i - 1], latitudes[i], longitudes[i])
13     return total_distance

```

W klasycznej implementacji Pythona (listing 3.3) wyznaczanie sumarycznego dystansu bazuje na pętli iterującej przez kolekcje danych. Funkcja `distance` wykorzystuje moduł

`math` do wyliczenia odległości między dwoma punktami ze wzoru Haversine’a. Następnie funkcja `total_distance` łączy kolejne pary punktów i dodaje wynik ich separacji do sumy całkowitej.

Listing 3.4: Funkcje odpowiedzialne za obliczanie dystansu (NumPy)

```

1 def distance_np(lat1, lon1, lat2, lon2):
2     phi1, phi2 = np.radians(lat1), np.radians(lat2)
3     delta_phi = np.radians(lat2 - lat1)
4     delta_lambda = np.radians(lon2 - lon1)
5     a = np.sin(delta_phi / 2.0) ** 2 + np.cos(phi1) * np.cos(
        phi2) * np.sin(delta_lambda / 2.0) ** 2
6     c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
7     return R * c
8
9 def total_distance_np(latitudes, longitudes):
10     distances = distance_np(latitudes[:-1], longitudes[:-1],
        latitudes[1:], longitudes[1:])
11     return np.sum(distances)

```

Chociaż implementacje tych algorytmów na bazie list języka Pythona oraz z wykorzystaniem biblioteki NumPy wyglądają na pierwszy rzut oka niemal identycznie, mechanika ich działania pod spodem jest zupełnie inna. Główna różnica sprowadza się do sposobu obsługi zmiennych. Standardowy moduł `math` w Pythonie operuje wyłącznie na pojedynczych wartościach. Z tego względu, aby przetworzyć kolekcję danych, konieczne jest zastosowanie jawnej pętli (`np. for`). Z kolei biblioteka NumPy wykorzystuje zjawisko wektoryzacji. Oznacza to, że funkcje takie jak `np.sin` czy `np.radians` mogą przyjmować jako argument całe tablice danych. Dzięki temu, mimo że linijki kodu z `math.sin` i `np.sin` wyglądają podobnie, ta druga potrafi przetworzyć miliony rekordów w ułamku sekundy.

W przypadku użycia NumPy (listing 3.4) logika została znacznie uproszczona. Funkcja `distance_np` zamiast pojedynczych współrzędnych, tak jak w przypadku `distance_np` (listing 3.3), przyjmuje przesunięte względem siebie wektory (odpowiednio od pierwszego do przedostatniego elementu oraz od drugiego do ostatniego). Eliminacja pętli `for` na rzecz `np.sum()` znacząco skraca czas wykonania i upraszcza architekturę kodu. W przypadku biblioteki NumPy (listing 3.4) logika została znacznie uproszczona. Operacje matematyczne wywoływane są na całych wektorach wejściowych bez potrzeby jawnej iteracji. Obliczenia przewyższeń (listing 3.4, linie 13-15) wykorzystują funkcję `np.diff()`, która automatycznie zwraca wektor różnic wysokości, a dzięki zastosowaniu maski logicznej `diffs > 0` funkcja błyskawicznie sumuje wyłącznie wartości dodatnie.

3.3.2 Obliczanie przewyższeń

Listing 3.5: Funkcje odpowiedzialne za obliczanie przewyższeń (listy, wektory)

```
1 def elevation_gain(elevations):
2     gain = 0.0
3     for e1, e2 in zip(elevations[:-1], elevations[1:]):
4         diff = e2 - e1
5         if diff > 0:
6             gain += diff
7     return gain
8
9 def elevation_gain_np(elevations):
10     diffs = np.diff(elevations)
11     return np.sum(diffs[diffs > 0])
```

Standardowe podejście do obliczania całkowitego przewyższenia (listing 3.5, linie 1-7) polega na sparowaniu kolejnych punktów wysokościowych za pomocą funkcji `zip()`, a następnie iteracyjnym obliczaniu różnic między nimi. Instrukcja warunkowa `if diff > 0` gwarantuje, że do ostatecznej sumy trafiają wyłącznie odcinki wznoszące (pomijane są zjazdy i odcinki płaskie).

W podejściu wektorowym (listing 3.5, linie 9-11) kod sprowadza się zaledwie do dwóch linijek. Wbudowana funkcja `np.diff()` automatycznie zwraca wektor różnic wysokości między wszystkimi sąsiadującymi elementami tablicy. W drugiej linii zastosowano maskę logiczną (`diffs > 0`). Wyrażenie to tworzy wektor wartości prawda/fałsz, który służy do odfiltrowania wyłącznie dodatnich różnic. Dopiero na tak przygotowanych danych wykonywana jest operacja sumowania `np.sum()`.

3.3.3 Analiza parametru tętna serca

Listing 3.6: Funkcje odpowiedzialne za analizę tętna

```
1 def avg_hr(heart_rates):
2     total_hr = 0
3     count = 0
4     for hr in heart_rates:
5         if hr > 0:
6             total_hr += hr
7             count += 1
8     return total_hr / count if count > 0 else 0
9
```

```

10 def hr_zones(heart_rates , hr_max=185):
11     z1_min, z2_min, z3_min, z4_min, z5_min = [hr_max * p for p
12         in (0.5, 0.6, 0.7, 0.8, 0.9)]
13     zones = [0, 0, 0, 0, 0]
14     for hr in heart_rates:
15         if hr >= z5_min: zones[4] += 1
16         elif hr >= z4_min: zones[3] += 1
17         elif hr >= z3_min: zones[2] += 1
18         elif hr >= z2_min: zones[1] += 1
19         elif hr >= z1_min: zones[0] += 1
20     return zones
21
22 def elevation_hr(elevations , heart_rates):
23     climb_hr_total, climb_count, descent_hr_total, descent_count
24     = 0, 0, 0, 0
25     for e1, e2, hr in zip(elevations[:-1], elevations[1:],
26         heart_rates[1:]):
27         ele_diff = e2 - e1
28         if hr > 0:
29             if ele_diff > 0:
30                 climb_hr_total += hr
31                 climb_count += 1
32             else:
33                 descent_hr_total += hr
34                 descent_count += 1
35     avg_climb = climb_hr_total / climb_count if climb_count > 0
36     else 0
37     avg_descent = descent_hr_total / descent_count if
38     descent_count > 0 else 0
39     return avg_climb, avg_descent

```

Pierwsza z funkcji, `avg_hr` (listing 3.6, linie 1–8), odpowiada za wyliczenie średniej wartości tętna z całej aktywności. Do całkowitej sumy oraz licznika pomiarów dodawane są wyłącznie wartości większe od zera.

Druga funkcja, `hr_zones` (listing 3.6, linie 9–19), ma za zadanie przyporządkować poszczególne odczyty do jednej z pięciu standardowych stref wysiłkowych, bazując na wartości tętna maksymalnego. Konstrukcja w linii 11 wylicza dolne progi tętna dla stref od Z1 do Z5, które stanowią od 50% do 90% tętna maksymalnego, a następnie przypisuje je do pięciu osobnych zmiennych. Z kolei w liniach 13–18 pętla iteruje po wszystkich

pomiarach, przydzielając je do odpowiednich kategorii.

Ostatnia funkcja, `elevation_hr` (listing 3.6, linie 20–34), analizuje obciążenie organizmu w kontekście profilu wysokościowego pokonywanej trasy. Jej głównym celem jest podział danych o tętnie na odcinki wznoszące się oraz opadające. W każdej iteracji pętli program uzyskuje bezpośredni dostęp do dwóch sąsiadujących pomiarów wysokości i powiązanego z nimi tętna. Funkcja wylicza różnicę wysokości między punktami, a następnie przypisuje wartość tętna do odpowiedniego licznika w zależności czy nastąpił przyrost wysokości bądź jej spadek. Na koniec zwracane są średnie wartości tętna dla obu rodzajów nachylenia terenu.

Listing 3.7: Funkcje odpowiedzialne za analizę tętna (NumPy)

```
1 def avg_hr_np(heart_rates):
2     valid_hr = heart_rates[heart_rates > 0]
3     return np.mean(valid_hr) if valid_hr.size > 0 else 0
4
5
6 def hr_zones_np(heart_rates, hr_max=185):
7     bins = [hr_max * 0.5, hr_max * 0.6, hr_max * 0.7, hr_max *
8             0.8, hr_max * 0.9, np.inf]
9     counts, _ = np.histogram(heart_rates, bins=bins)
10    return counts.tolist()
11
12 def elevation_hr_np(elevations, heart_rates):
13     ele_diffs = np.diff(elevations)
14     hr_segments = heart_rates[1:]
15     valid_hr = hr_segments > 0
16     is_climb = ele_diffs > 0
17     is_descent_or_flat = ele_diffs <= 0
18     hr_on_climbs = hr_segments[valid_hr & is_climb]
19     hr_on_descents = hr_segments[valid_hr & is_descent_or_flat]
20     avg_climb = np.mean(hr_on_climbs) if hr_on_climbs.size > 0
21     else 0
22     avg_descent = np.mean(hr_on_descents) if hr_on_descents.size
23     > 0 else 0
24     return avg_climb, avg_descent
```

W pierwszej funkcji, `avg_hr_np` (listing 3.7, linie 1–3), proces filtrowania danych i obliczania średniej uległ znacznemu uproszczeniu. Druga z funkcji, `hr_zones_np` (listing 3.7, linie 6–9), kompletnie usunęła kaskadę instrukcji warunkowych. Zamiast tego, w linii

7 zdefiniowano tablicę bins, określającą krawędzie przedziałów dla poszczególnych stref tętna. Górną granicę ostatniej strefy (Z5) wyznaczono tu jako `np.inf`, aby przechwycić najwyższe odczyty. W linii 8, gdzie użyto funkcji `np.histogram()`. Przeprowadza ona grupowanie wszystkich wartości z tablicy `heart_rates` do zdefiniowanych przedziałów i zwraca gotową tablicę liczników wystąpień dla każdej strefy. Wynik ten jest konwertowany na standardową listę za pomocą metody `tolist()`.

Ostatnia funkcja, `elevation_hr_np` (listing 3.7, linie 12–22), została znacząco zmieniona względem standardowej implementacji. Funkcja `np.diff` automatycznie wylicza różnice między każdym elementem tablicy wysokości oraz jego sąsiadem. Następnie zdefiniowano zmienne przechowujące maski logiczne o poprawności tętna (`valid_hr`) oraz kierunku nachylenia trasy (`is_climb`, `is_descent_or_flat`). W liniach 18 i 19 te maski są nakładane na siebie przy użyciu operatora `&`.

3.3.4 Analiza wydolności biomechanicznej i tlenowej

Funkcja `perf_eff` (listing 3.8) to narzędzie analityczne, które pozwala na oszacowanie wydolności biomechanicznej i tlenowej. Kod ten został podzielony na trzy główne etapy: estymację mocy chwilowej, obliczenie mocy znormalizowanej oraz ocenę zmęczenia (rozprężenia tlenowego).

Listing 3.8: Analiza wydolności biomechanicznej i tlenowej

```

1 def perf_eff(latitudes , longitudes , elevations , heart_rates ,
    times , weight=75.0):
2     delta_times = []
3     distances = []
4     speeds = []
5     grades = []
6     powers = []
7
8     for i in range(1, len(latitudes)):
9         delta_time = times[i] - times[i - 1]
10        if delta_time <= 0:
11            delta_time = 1.0
12
13        dist = distance(latitudes[i - 1], longitudes[i - 1],
            latitudes[i], longitudes[i])
14        ele_diff = elevations[i] - elevations[i - 1]
15        speed = dist / delta_time
16        grade = (ele_diff / dist) if dist > 0 else 0

```

```
17         power = (weight * 9.81 * speed * grade) + (0.5 * weight
18                 * speed)
19     power = max(0, power)
20
21     delta_times.append(delta_time)
22     distances.append(dist)
23     speeds.append(speed)
24     grades.append(grade)
25     powers.append(power)
26
27     power_4th = []
28     window_size = 30
29     for i in range(len(powers)):
30         start = max(0, i - window_size + 1)
31         window = powers[start:i + 1]
32         rolling_mean = sum(window) / len(window)
33         power_4th.append(rolling_mean ** 4)
34
35     normalized_power = (sum(power_4th) / len(power_4th)) ** 0.25
36     if power_4th else 0
37     half = len(powers) // 2
38
39     def calculate_ef(power, hr):
40         avg_p = sum(power) / len(power) if power else 0
41         avg_hr = sum(hr) / len(hr) if hr else 1
42         return avg_p / avg_hr
43
44     hr_data = heart_rates[1:]
45
46     ef_1 = calculate_ef(powers[:half], hr_data[:half])
47     ef_2 = calculate_ef(powers[half:], hr_data[half:])
48     decoupling = ((ef_1 - ef_2) / ef_1 * 100) if ef_2 > 0 else 0
49
50     return {
51         "Znormalizowana moc (W)": round(normalized_power, 2),
52         "Współczynnik wydajności (pierwsza połowa treningu)":
53             round(ef_1, 3),
54         "Współczynnik wydajności (druga połowa treningu)": round
```

```

52         (ef_2 , 3) ,
53         "Aerobic decoupling": round(decoupling , 2)
54     }

```

Pierwszy etap odbywa się w pętli głównej (od linii 15). Program iteruje po wszystkich zapisanych punktach trasy, obliczając dla każdego kroku czas trwania, pokonany dystans, zmianę wysokości, prędkość oraz nachylenie terenu. Linia 28 odpowiada za estymację generowanej mocy. Wykorzystano uproszczony model matematyczny składający się z dwóch członów. Pierwszy z nich (`weight * 9.81 * speed * grade`) odpowiada za pracę przeciwko grawitacji na podjazdach i podbiegach. Drugi człon (`0.5 * weight * speed`) to szacunkowy koszt pokonywania oporów ruchu. W linii 29 zastosowano funkcję `max(0, power)`, aby zabezpieczyć funkcję przed wyliczeniem ujemnej mocy podczas stromych zjazdów.

Drugi etap (linie 26–34) to wyliczenie mocy znormalizowanej, która odzwierciedla fizjologiczny koszt wysiłku. W liniach 28–32 zastosowano algorytm średniej kroczącej z 30-sekundowym oknem czasowym, aby wygładzić chwilowe skoki mocy. Podniesienie każdej uśrednionej wartości do czwartej potęgi sprawia, że krótkie, ale intensywne momenty wysiłku otrzymują znacznie większą wagę w ostatecznym rozrachunku. Cały proces kończy się w linii 46 wyciągnięciem pierwiastka czwartego stopnia z sumy tych wartości, przywracając wynik do skali w watach.

Ostatni etap to analiza wskaźnika rozprężenia tlenowego (listing 3.8, linie 35–53). W tym celu została zdefiniowana funkcja pomocnicza `calculate_ef`, która wylicza współczynnik wydajności, który mówi o tym, ile watów mocy organizm generuje średnio na jedno uderzenie serca. Następnie dane z treningu dzielone są na dwie równe połowy. Obliczenie wydajności dla pierwszej i drugiej połowy pozwala na ich bezpośrednie zestawienie. W linii 46 wyliczany jest procentowy spadek tej wydajności w czasie. Jeśli tętno w drugiej połowie rośnie przy utrzymaniu tej samej mocy (lub moc spada przy tym samym tętnie), wskaźnik decoupling rośnie, co świadczy o narastającym zmęczeniu układu sercowo-naczyniowego. Całość zwracana jest w formie słownika z zaokrąglonymi wartościami.

Listing 3.9: Analiza wydolności biomechanicznej i tlenowej (NumPy)

```

1 def perf_eff_np(latitudes , longitudes , elevations , heart_rates ,
2   delta_time = np.diff(times)
3   delta_time = np.maximum(delta_time , 1.0)
4
5   dist = distance_np(latitudes[:-1] , longitudes[:-1] ,
6     latitudes[1:] , longitudes[1:])
7   ele_diff = np.diff(elevations)

```

```

7     speed = dist / delta_time
8
9     grade = np.divide(ele_diff, dist, out=np.zeros_like(ele_diff),
10                      where=(dist > 0))
11
12     power = (weight * 9.81 * speed * grade) + (0.5 * weight *
13           speed)
14     power = np.maximum(0, power)
15
16     window_size = 30
17     window = np.ones(window_size) / window_size
18     rolling_mean = np.convolve(power, window, mode='valid')
19     normalized_power = np.mean(rolling_mean ** 4) ** 0.25 if len
20         (rolling_mean) > 0 else 0.0
21
22     half = len(power) // 2
23     pow_1, pow_2 = power[:half], power[half:]
24
25     hr_data = heart_rates[1:]
26     hr_1, hr_2 = hr_data[:half], hr_data[half:]
27
28     avg_hr_1 = np.mean(hr_1) if hr_1.size > 0 else 0
29     avg_hr_2 = np.mean(hr_2) if hr_2.size > 0 else 0
30
31     ef_1 = np.mean(pow_1) / avg_hr_1 if avg_hr_1 > 0 else 0.0
32     ef_2 = np.mean(pow_2) / avg_hr_2 if avg_hr_2 > 0 else 0.0
33     decoupling = ((ef_1 - ef_2) / ef_1 * 100) if ef_1 > 0 else
34         0.0
35
36     return {
37         "Znormalizowana moc (W)": round(float(normalized_power),
38             2),
39         "Współczynnik wydajności (pierwsza połowa treningu)":
40             round(float(ef_1), 3),
41         "Współczynnik wydajności (druga połowa treningu)": round
42             (float(ef_2), 3),
43         "Aerobic decoupling": round(float(decoupling), 2)
44     }

```

W pierwszej kolejności wektoryzacji uległy podstawowe parametry fizyczne. Ręczne wyliczanie różnic czasu przeniesiono na funkcję `np.diff()`, a blokady przed wartościami ujemnymi zrealizowano za pomocą funkcji `np.maximum()`. Prędkość i dystans są teraz wyliczane jednorazowo dla całych tablic dzięki mechanizmowi krojenia list. Dodatkowo, klasyczne instrukcje warunkowe zapobiegające dzieleniu przez zero przy obliczaniu nachylenia terenu zastąpiono dzieleniem za pomocą funkcji `np.divide`. Wykorzystano w niej parametry `where`, który nakazuje dzielenie tylko dla dystansu większego od zera, oraz `out`, który automatycznie wstawia zera w pozostałych przypadkach.

Najbardziej znacząca zmiana zaszła w mechanizmie obliczania mocy znormalizowanej. Iterowanie po 30-sekundowym oknie czasowym całkowicie wyeliminowano na rzecz wbudowanej funkcji `np.convolve()`. Działa ona jak ruchome okno, które automatycznie przesuwa się po wszystkich odczytach mocy. Osiągamy w ten sposób identyczny efekt jak w przypadku tradycyjnej średniej kroczącej.

Znacznemu skróceniu uległa również końcowa analiza rozprężenia tlenowego. Usunięto wewnętrzną funkcję pomocniczą, a tablice mocy i tętna są teraz bezpośrednio dzielone na pół za pomocą indeksowania (wycinki `[:half]` oraz `[half:]`). Wartości wydajności wyliczane są od razu dla całych podzbiorów przez funkcję `np.mean()`.

3.3.5 Segmentacja aktywności podczas treningu

W celu wyodrębnienia ciągłych etapów aktywności, takich jak chód, bieg czy postój, z surowych danych telemetrycznych, opracowano algorytm oparty na iteracyjnej analizie szeregów czasowych. Implementację tego rozwiązania podzielono na trzy główne etapy: przygotowanie i klasyfikację danych, filtrowanie i scalanie segmentów oraz agregację wyników końcowych.

Listing 3.10: Klasyfikacja stanów aktywności

```

1 def activity_segments(latitudes , longitudes , times , cadences ,
    min_duration=5):
2     n = len(times)
3     dist = []
4     for i in range(n - 1):
5         distance = distance(latitudes[i] , longitudes[i] ,
            latitudes[i + 1] , longitudes[i + 1])
6         dist.append(distance)
7     delta_time = []
8     for i in range(n - 1):
9         dt = times[i + 1] - times[i]
10        delta_time.append(dt if dt != 0 else 1.0)
11    speeds = [0.0]
```

```
12     for i in range(n - 1):
13         speeds.append(dist[i] / delta_time[i])
14
15     states = []
16     for i in range(n):
17         speed = speeds[i]
18         cadence = cadences[i]
19         state = 0
20         if (0.5 < speed <= 2.2) or (0 < cadence < 65):
21             state = 1
22         if speed > 2.2 and cadence >= 65:
23             state = 2
24         if 0.5 < speed < 3.0 and cadence == 0:
25             state = 0
26         states.append(state)
27
28     split_indices = [0]
29     for i in range(1, n):
30         if states[i] != states[i - 1]:
31             split_indices.append(i)
32     split_indices.append(n)
```

Pierwszy etap algorytmu (listing 3.10) rozpoczyna się od wyliczenia podstawowych parametrów fizycznych dla każdej zebranej próbki danych. W liniach 4–10 program iteruje po tablicach z danymi, obliczając dystanse między kolejnymi punktami za pomocą funkcji `distance`, która była omówiona w poprzednim podrozdziale oraz różnice czasu pomiędzy odczytami (`delta_time`). W linii 10 zastosowano mechanizm zapobiegający błędowi dzielenia przez zero. W przypadku braku różnicy w czasie pomiaru, wartość ta jest sztucznie ustawiana na 1.0. Następnie w pętli (listing 3.10, linie 12–14) wyznaczane są prędkości wyrażone w metrach na sekundę.

Następnie od linii 17 zaczyna się klasyfikacja punktów. Algorytm iteruje po listach z prędkościami oraz kadencjami. Na podstawie analizy warunkowej instrukcjami `if`, przypisywany jest jeden z trzech stanów: postój (0), chód (1) lub bieg (2). Chód jest identyfikowany, gdy prędkość mieści się w przedziale od 0.5 do 2.2 m/s lub gdy urządzenie rejestruje niewielką kadencję poniżej 65 kroków na minutę. Bieg wymaga spełnienia przekroczenia warunków obu tych wartości (linia 23). Dodatkowo, w liniach 25–26 uwzględniono przypadek, w którym pomimo rejestrowania prędkości z modułu GPS, sensor krokomierza zwraca wartość zerową. Taka sytuacja traktowana jest jako postój lub dryf sygnału. Ostatnim krokiem jest odnalezienie momentów zmian aktywności. W liniach 30–32 pętla

sprawdza, czy stan w obecnym kroku różni się od stanu z kroku poprzedniego, i zapisuje te indeksy w liście `split_indices`.

Listing 3.11: Dzielenie segmentów aktywności (Python)

```

1  segments = []
2  for i in range(len(split_indices) - 1):
3      start_index = split_indices[i]
4      end_index = split_indices[i + 1]
5      state = states[start_index]
6      segments.append({
7          'state': state,
8          'start_idx': start_index,
9          'end_idx': end_index,
10         'duration': end_index - start_index
11     })
12
13  filtered_segments = []
14  for segment in segments:
15      if segment['duration'] < min_duration and len(
16         filtered_segments) > 0:
17         segment['state'] = filtered_segments[-1]['state']
18         filtered_segments.append(segment)
19
20  final_segments = []
21  for segment in filtered_segments:
22      if not final_segments:
23         final_segments.append(segment)
24      elif final_segments[-1]['state'] == segment['state']:
25         final_segments[-1]['end_idx'] = segment['end_idx']
26         final_segments[-1]['duration'] += segment['duration']
27     ]
28
29  else:
30     final_segments.append(segment)
31
32  state_names = {0: "Postój (Idle)", 1: "Chód (Walk)", 2: "
    Bieg (Run)"}

```

Listing 3.11 obrazuje proces transformacji surowych punktów podziału w logiczne i wygładzone bloki aktywności. W liniach 2–10 indeksy ze `split_indices` są mapowane na listę słowników `segments`, z których każdy przechowuje informacje o swoim stanie,

granicach indeksów oraz czasie trwania. Ponieważ odczyty z sensorów mogą charakteryzować się krótkotrwałymi zakłóceniami (np. chwilowe zatrzymanie na światłach podczas biegu), wprowadzono mechanizm filtracji. Pętla w liniach 13–16 sprawdza, czy dany segment jest krótszy niż założony parametr `min_duration`. Jeśli tak, segment ten przejmuje stan od aktywności poprzedzającej. Przefiltrowane dane wymagają ponownego ujednolicenia struktury. Odpowiada za to pętla w liniach 19–25, która iteruje po przefiltrowanych danych i łączy sąsiadujące bloki o tym samym stanie w dłuższe, ciągłe segmenty (`final_segments`), sumując ich czas trwania i aktualizując indeksy końcowe.

Listing 3.12: Formatowanie danych wyjściowych (Python)

```
1  results = []
2
3  for segment in final_segments:
4      start_index = segment['start_idx']
5      end_index = segment['end_idx'] - 1
6
7      if start_index >= end_index:
8          continue
9
10     start_time_str = datetime.fromtimestamp(times[
11         start_index]).strftime('%H:%M:%S')
12     end_time_str = datetime.fromtimestamp(times[end_index]).
13         strftime('%H:%M:%S')
14
15     segment_dist = sum(dist[start_index:end_index])
16     speed_slice = speeds[start_index:end_index + 1]
17     avg_speed_kmh = (sum(speed_slice) / len(speed_slice)) *
18         3.6 if speed_slice else 0.0
19     cadence_slice = cadences[start_index:end_index + 1]
20     avg_cadence = sum(cadence_slice) / len(cadence_slice) if
21         cadence_slice else 0.0
22
23     results.append({
24         'Typ': state_names[segment['state']],
25         'Start': start_time_str,
26         'Koniec': end_time_str,
27         'Dystans (m)': round(segment_dist, 1),
28         'Średnia prędkość (km/h)': round(avg_speed_kmh, 1),
29         'Średnia kadencja (RPM)': int(round(avg_cadence, 0))
30     })
```

```

26         * 2
27     })
28     return results

```

Ostatni fragment funkcji (listing 3.12) odpowiada za formatowanie danych wyjściowych do postaci czytelnej dla użytkownika. W liniach 3–23 program iteruje po ostatecznych segmentach z listy `final_segments`. Z użyciem biblioteki `datetime` (linie 8–9) indeksy czasowe zamieniane są na format znakowy przedstawiający godzinę (%H:%M:%S). Następnie algorytm wyodrębnia wycinki z oryginalnych tablic dystansu, prędkości i kadencji za pomocą indeksów początkowych i końcowych danego bloku. Na ich podstawie, wykorzystując wbudowane funkcje `sum()` i `len()`, obliczane są uśrednione wartości dla całego segmentu (linie 11–15). Średnia prędkość jest przeliczana na bardziej intuicyjne kilometry na godzinę. Zgromadzone statystyki trafiają do listy `results` jako sformatowane i zaokrąglone wartości, tworząc raport podsumowujący poszczególne etapy treningu.

Listing 3.13: Segmentacja aktywności podczas treningu (NumPy)

```

1 def activity_segments_np(latitudes, longitudes, times, cadences,
2   min_duration=5):
3     delta_time = np.diff(times)
4     delta_time[delta_time == 0] = 1.0
5     dist = distance_np(latitudes[:-1], longitudes[:-1],
6       latitudes[1:], longitudes[1:])
7     speeds = np.insert(dist / delta_time, 0, 0.0)
8
9     states = np.zeros(len(times), dtype=int)
10    walk_mask = ((speeds > 0.5) & (speeds <= 2.2)) | ((cadences
11      > 0) & (cadences < 65))
12    states[walk_mask] = 1
13    run_mask = (speeds > 2.2) & (cadences >= 65)
14    states[run_mask] = 2
15    idle_mask = (speeds > 0.5) & (cadences == 0) & (speeds <
16      3.0)
17    states[idle_mask] = 0
18
19    changes = np.where(states[:-1] != states[1:])[0] + 1
20    split_indices = np.concatenate([0, changes, [len(states)]]
21      )
22
23    # [...]

```

```
19     # Proces tworzenia słowników 'segments', filtrowania na
20     # podstawie
21     # 'min_duration' i scalania przyległych bloków w '
22     # final_segments'
23     # pozostaje iteracyjny i identyczny z rozwiązaniem opartym
24     # na listach Pythona.
25     # [...]
26     results = []
27     for segment in final_segments:
28         start_index = segment['start_idx']
29         end_index = segment['end_idx'] - 1
30         start_time_str = datetime.fromtimestamp(times[
31             start_index]).strftime('%H:%M:%S')
32         end_time_str = datetime.fromtimestamp(times[end_index]).
33             strftime('%H:%M:%S')
34
35         if start_index >= end_index:
36             continue
37
38         segment_dist = np.sum(dist[start_index:end_index])
39         avg_speed_kmh = (np.mean(speeds[start_index:end_index +
40             1])) * 3.6
41         avg_cadence = np.mean(cadences[start_index:end_index +
42             1])
43
44         results.append({
45             'Typ': state_names[segment['state']],
46             'Start': start_time_str,
47             'Koniec': end_time_str,
48             'Dystans (m)': round(segment_dist, 1),
49             'Średnia prędkość (km/h)': round(avg_speed_kmh, 1),
50             'Średnia kadencja (RPM)': int(round(avg_cadence, 0))
51             *2
52         })
53     return results
```

Działanie algorytmu rozpoczyna się od wyznaczenia podstawowych parametrów fizycznych. Zamiast iteracyjnie odejmować kolejne znaczniki czasu w pętli (listing 3.10, linie 8–10), wykorzystano tu wektorową funkcję `np.diff`, która w jednym kroku oblicza różnice dla całej trasy. Aby uniknąć błędu dzielenia przez zero, tablica jest modyfikowana za pomocą indeksowania warunkowego (listing 3.13, linia 3), co stanowi zamiennik dla instrukcji `if/else` (listing 3.10, linia 10). Dystans wyliczany jest przy użyciu funkcji `distance_np`, do której trafiają przesunięte względem siebie wycinki tablic współrzędnych. Pozwala to na jednoczesne przeliczenie całej ścieżki i eliminuje konieczność wywoływania funkcji dla każdego punktu z osobna (listing 3.10, linie 4–6). Z kolei do wyrównania długości tablicy prędkości, uzyskanej z podzielenia wektorów dystansu przez czas, wykorzystano funkcję `np.insert`, co zastępuje iteracyjne dodawanie elementów do nowej listy (listing 3.10, linie 12–14).

Najistotniejszą zmianą w stosunku do klasycznego podejścia iteracyjnego jest mechanizm klasyfikacji stanów aktywności. Zamiast rozbudowanych instrukcji warunkowych, weryfikowanych wielokrotnie dla każdej próbki na przestrzeni kilkunastu linii kodu (listing 3.10, linie 17–27), algorytm tworzy początkowo zerową tablicę stanów (listing 3.13, linia 7). Następnie definiuje maski logiczne przy pomocy wektorowych operatorów bitowych koniunkcji i alternatywy. Ewaluacja złożonych warunków progowych generuje struktury składające się z wartości logicznych prawdy i fałszu. Pozwala to w pojedynczych instrukcjach indeksowania przypisać odpowiedni stan, taki jak chód, bieg czy postój, do wszystkich pasujących indeksów jednocześnie. W analogiczny sposób, całkowicie eliminując pętlę porównującą stany sąsiadujące (listing 3.10, linie 30–32), lokalizowane są momenty zmian aktywności. Wyrażenie `states[:-1] != states[1:]` porównuje tablicę z jej przesuniętą kopią, a funkcja `np.where` wyciąga indeksy różnic, które na koniec są łączone z początkiem i końcem trasy za pomocą `np.concatenate`, tworząc kompletną listę granic segmentów.

Jak zaznaczono w komentarzach wewnątrz listingu, ze względu na sekwencyjny charakter operacji, procesy mapowania indeksów na słowniki, filtrowania zbyt krótkich aktywności oraz łączenia przyległych bloków o tym samym stanie zachowują logikę identyczną z wersją opartą na wbudowanych strukturach języka Python. W końcowym etapie kod przechodzi do agregacji i formatowania wyników, gdzie iteruje po ostatecznych, scalonych segmentach. Indeksy brzegowe wykorzystywane są do konwersji surowych znaczników czasu na czytelny format znakowy za pomocą wbudowanego modułu `datetime` oraz do wyciągania wycinków z utworzonych wcześniej tablic NumPy. Finalne statystyki segmentu obliczane są z użyciem wydajnych funkcji agregujących: `np.sum` do zsumowania dystansu oraz `np.mean` do wyliczenia średniej prędkości z przeliczeniem na kilometry na godzinę oraz uśrednienia kadencji. Zgromadzone i zaokrąglone dane trafiają do listy, tworząc ustrukturyzowany raport podsumowujący cały trening.

Rozdział 4

Przeprowadzenie analizy porównawczej

Ten rozdział poruszy zagadnienia związane z analizą wydajności zaimplementowanych rozwiązań. W pierwszej kolejności zdefiniowane zostanie środowisko testowe, a także krótki opis specyfikacji sprzętowej. W dalszej części opisane zostaną szczegóły implementacyjne narzędzi badawczych: generatora sztucznych plików TCX oraz programu do benchmarku czasowego. Następnie przedstawione zostaną wyniki pomiarów czasu wykonania, a także ich interpretacja. Ostatnia część rozdziału zostanie poświęcona na badanie chybień pamięci podręcznej procesora. Przedstawiono w niej proces badawczy oraz drogę do wypracowania metodyki pozwalającej na pomiar tych wskaźników w środowisku wirtualnym.

4.1 Specyfikacja sprzętowa

Wiarygodność oraz powtarzalność wszelkich pomiarów wydajnościowych wymaga zdefiniowania środowiska, w którym zostały one przeprowadzone. Nawet niewielkie różnice w architekturze procesora, rozmiarze pamięci podręcznej czy wersji kompilatora mogą znacząco wpłynąć na ostateczne wyniki czasowe oraz profilowanie sprzętowe. Zestawienie najważniejszych parametrów specyfikację platformy sprzętowej oraz środowiska testowego przedstawiono w tabeli 4.1. Badania zrealizowano z wykorzystaniem nowoczesnego, sześciordzeniowego procesora. Należy zwrócić szczególną uwagę na architekturę pamięci pierwszego poziomu, która jest fizycznie rozdzielona na pamięć instrukcji (L1i) oraz pamięć danych (L1d). Każda z nich dysponuje łączną pojemnością 192 KiB. Braki na tym poziomie są następnie kompensowane przez pojemniejszą pamięć L2 (3 MB) oraz stosunkowo dużą, współdzieloną pamięć L3 (16 MB).

Testy przeprowadzono z wykorzystaniem architektury WSL 2 (ang. *Windows Subsystem for Linux*). Zastosowanie wirtualizacji systemu Ubuntu jako środowiska gościa

umożliwiło dostęp do zaawansowanych, linuksowych narzędzi profilujących. Wszystkie skrypty testowe i algorytmy były uruchamiane pod kontrolą interpretera języka Python w wersji 3.13.

Tabela 4.1: Specyfikacja środowiska sprzętowo-programowego

Komponent / Oprogramowanie	Specyfikacja
Procesor (CPU)	AMD Ryzen 5 7535HS (6 rdzeni, 12 wątków, 3.30 GHz)
Pamięć Cache procesora	L1d/L1i: 192 KiB, L2: 3.0 MB, L3: 16.0 MB
Pamięć operacyjna (RAM)	16 GB SODIMM (4800 MT/s)
Pamięć masowa (dysk)	WD PC SN5000S 512GB (NVMe SSD)
System operacyjny	Microsoft Windows 11 Home (Kompilacja 26200)
Środowisko testowe	WSL 2 – Ubuntu 24.04.4 LTS (Noble Numbat)
Wersja interpretera Pythona	Python 3.13

4.2 Przygotowanie danych i implementacja narzędzi badawczych

W tej sekcji omówiony zostanie proces przygotowania środowiska badawczego od strony programistycznej i metodologicznej. Zanim możliwe było przeprowadzenie właściwych pomiarów, konieczne było przygotowanie ustandaryzowanych zbiorów danych testowych oraz implementacja odpowiednich mechanizmów badawczych do monitorowania działania kodu.

4.2.1 Generowanie danych testowych

Aby zapewnić miarodajne i powtarzalne wyniki testów wydajnościowych, konieczne było pozyskanie odpowiednio dużych, kontrolowanych zbiorów danych. W tym celu zaimplementowano dedykowany program, którego zadaniem jest sztuczne generowanie plików w formacie TCX. Skrypt pozwala na stworzenie pliku o dowolnie wybranej przez użytkownika liczbie punktów pomiarowych, a gotowe zbiory zapisywane są bezpośrednio w docelowym katalogu z danymi badawczymi.

Listing 4.1: Generator syntetycznych danych treningowych w formacie TCX

```

1 def generate(filename=" ../data/plik_1000000.tcx", num_points=100
  _000):
2     start_time = datetime.datetime.now(datetime.timezone.utc)
3     center_lat = 52.0

```

```

4     center_lon = 19.0
5     radius = 0.5
6
7     with open(filename, "w", encoding="utf-8") as f:
8         f.write('<?xml version="1.0" encoding="UTF-8"?>\n')
9         f.write('<TrainingCenterDatabase\n')
10        f.write('  xsi:schemaLocation="http://www.garmin.com/
        xmlschemas/TrainingCenterDatabase/v2 http://www.
        garmin.com/xmlschemas/TrainingCenterDatabasev2.xsd"\n
        ')
11        f.write('  xmlns:ns5="http://www.garmin.com/xmlschemas/
        ActivityGoals/v1"\n')
12        f.write('  xmlns:ns3="http://www.garmin.com/xmlschemas/
        ActivityExtension/v2"\n')
13        f.write('  xmlns:ns2="http://www.garmin.com/xmlschemas/
        UserProfile/v2"\n')
14        f.write('  xmlns="http://www.garmin.com/xmlschemas/
        TrainingCenterDatabase/v2"\n')
15        f.write('  xmlns:xsi="http://www.w3.org/2001/XMLSchema-
        instance"\n')
16        f.write('  xmlns:ns4="http://www.garmin.com/xmlschemas/
        ProfileExtension/v1">\n')
17        f.write('    <Activities>\n')
18        f.write('      <Activity Sport="Biking">\n')
19        f.write(f'        <Id>{start_time.strftime("%Y-%m-%dT%H:%M
        :%SZ")}</Id>\n')
20        f.write('        <Lap StartTime="{}">\n'.format(start_time
        .strftime("%Y-%m-%dT%H:%M:%SZ")))
21        f.write('          <Track>\n')
22
23        for i in range(num_points):
24            current_time = start_time + datetime.timedelta(
                seconds=i)
25
26            angle = (i % 100_000) / 100_000 * 2 * math.pi
27            lat = center_lat + math.sin(angle) * radius
28            lon = center_lon + math.cos(angle) * radius
29

```

```

30         ele = 200 + math.sin(i / 5000) * 100 + random.
           uniform(-1, 1)
31
32         hr = int(145 + math.sin(i / 2000) * 20 + random.
           randint(-2, 2))
33         cadence = int(90 + random.randint(-5, 5))
34
35         trackpoint = f"""                <Trackpoint>
36         <Time>{current_time.strftime("%Y-%m-%dT%H:%M:%SZ")}
           }</Time>
37         <Position>
38             <LatitudeDegrees>{lat:.6f}</LatitudeDegrees>
39             <LongitudeDegrees>{lon:.6f}</LongitudeDegrees>
40         </Position>
41         <AltitudeMeters>{ele:.1f}</AltitudeMeters>
42         <HeartRateBpm>
43             <Value>{hr}</Value>
44         </HeartRateBpm>
45         <Extensions>
46             <ns3:TPX>
47                 <ns3:RunCadence>{cadence}</ns3:RunCadence>
48             </ns3:TPX>
49         </Extensions>
50     </Trackpoint>\n"""
51
52     f.write(trackpoint)
53
54     if (i + 1) % 100_000 == 0:
55         print(f"Wygenerowano {i + 1} / {num_points}
           punktów...")
56
57     f.write('                </Track>\n')
58     f.write('            </Lap>\n')
59     f.write('        </Activity>\n')
60     f.write('    </Activities>\n')
61     f.write('</TrainingCenterDatabase>\n')
62     print(f"Zakończono! Plik {filename} jest gotowy do analizy.")
    )

```


W celu zachowania dokładności testów, wewnętrzna struktura i schemat XML generowanych dokumentów zostały zaczerpnięte bezpośrednio z autentycznych danych, zarejestrowanych przez zegarek sportowy Garmin Forerunner 55. Zapewnia to wierne odwzorowanie układu węzłów i typów danych, które znajdują się w prawdziwych danych treningowych. Wygenerowane w ten sposób pliki o zróżnicowanym rozmiarze posłużyły w kolejnych etapach do przeprowadzenia analizy czasu wykonania algorytmów oraz badania zjawiska chybieni pamięci podręcznej (ang. *cache misses*).

4.2.2 Program do analizy czasu wykonania algorytmów analitycznych

Aby móc obiektywnie porównać wydajność obu zaimplementowanych podejść (bazującego na wbudowanych strukturach języka Python oraz wykorzystującego bibliotekę NumPy), przygotowano dedykowany skrypt benchmarkowy. Jego głównym zadaniem jest zautomatyzowane przeprowadzenie serii testów dla wygenerowanych wcześniej plików TCX o różnym wolumenie danych, a następnie agregacja wyników do ujednoliconego formatu.

Listing 4.2: Porównanie czasu wykonania algorytmów

```
1 def benchmark(files, output_file="../benchmark.json"):
2     results = []
3
4     for file in files:
5         print(f"Przetwarzanie pliku: {file}...")
6         latitudes, longitudes, elevations, heart_rates, times,
           cadences = parser_py(file)
7         points = len(latitudes)
8         latitudes_np, longitudes_np, elevations_np,
           heart_rates_np, times_np, cadences_np = parser_np(
               file)
9
10        start_p = time.perf_counter()
11        total_distance(latitudes, longitudes)
12        elevation_gain(elevations)
13        avg_hr(heart_rates)
14        hr_zones(heart_rates)
15        elevation_hr(elevations, heart_rates)
16        perf_eff(latitudes, longitudes, elevations, heart_rates,
               times)
```

```

17         activity_segments(latitudes , longitudes , times , cadences
18         )
19
20         time_p = time.perf_counter() - start_p
21
22         start_n = time.perf_counter()
23         total_distance_np(latitudes_np , longitudes_np)
24         elevation_gain_np(elevations_np)
25         avg_hr_np(heart_rates_np)
26         hr_zones_np(heart_rates_np)
27         elevation_hr_np(elevations_np , heart_rates_np)
28         perf_eff_np(latitudes_np , longitudes_np , elevations_np ,
29                     heart_rates_np , times_np)
30         activity_segments_np(latitudes_np , longitudes_np ,
31                             times_np , cadences_np)
32         time_n = time.perf_counter() - start_n
33
34         results.append({
35             "plik": file ,
36             "punkty": points ,
37             "czas_python": time_p ,
38             "czas_numpy": time_n ,
39             "przyspieszenie": time_p/time_n
40         })
41
42     with open(output_file , 'w' , encoding='utf-8') as f:
43         json.dump(results , f , indent=4)
44
45     print(f"\nGotowe! Zapisano wyniki do pliku: {output_file}")

```

Najważniejszym założeniem metodologicznym podczas projektowania skryptu była ścisła izolacja czasu obliczeń analitycznych od czasu operacji przetwarzania plików TCX. Z tego względu, jak zaprezentowano na listingu 4.2, proces parsowania plików (`parser_py` oraz `parser_np`) odbywa się poza mierzonym blokiem. Dzięki temu uzyskane wyniki odzwierciedlają rzeczywistą wydajność procesora i operacji na pamięci eliminując narzut związany z przepustowością dysku twardego.

Do pomiaru czasu wykorzystano funkcję `time.perf_counter()` z wbudowanej biblioteki Pythona. Skrypt sekwencyjnie uruchamia pełny zestaw funkcji analitycznych, najpierw w wariancie opartym na listach Python, a następnie z wykorzystaniem wektoryzacji NumPy. Oprócz bezwzględnych czasów wykonania, dla każdego pliku obliczany

jest bezpośredni współczynnik przyspieszenia (ang. *speedup*). Zgromadzone statystyki są na koniec eksportowane do formatu JSON, co umożliwia ich łatwą wizualizację na późniejszym etapie analizy.

4.3 Analiza czasu wykonania algorytmów analitycznych

W celu weryfikacji wydajności obu zaimplementowanych rozwiązań, przeprowadzono testy na sztucznie wygenerowanych plikach TCX o rosnącym rozmiarze danych – od 1 000 do 1 000 000 punktów pomiarowych. Otrzymane wyniki, stanowiące wypadkową czasów wykonania całego zestawu analitycznego, zestawiono w tabeli 4.2.

Tabela 4.2: Zestawienie czasów wykonania algorytmów analitycznych (w sekundach)

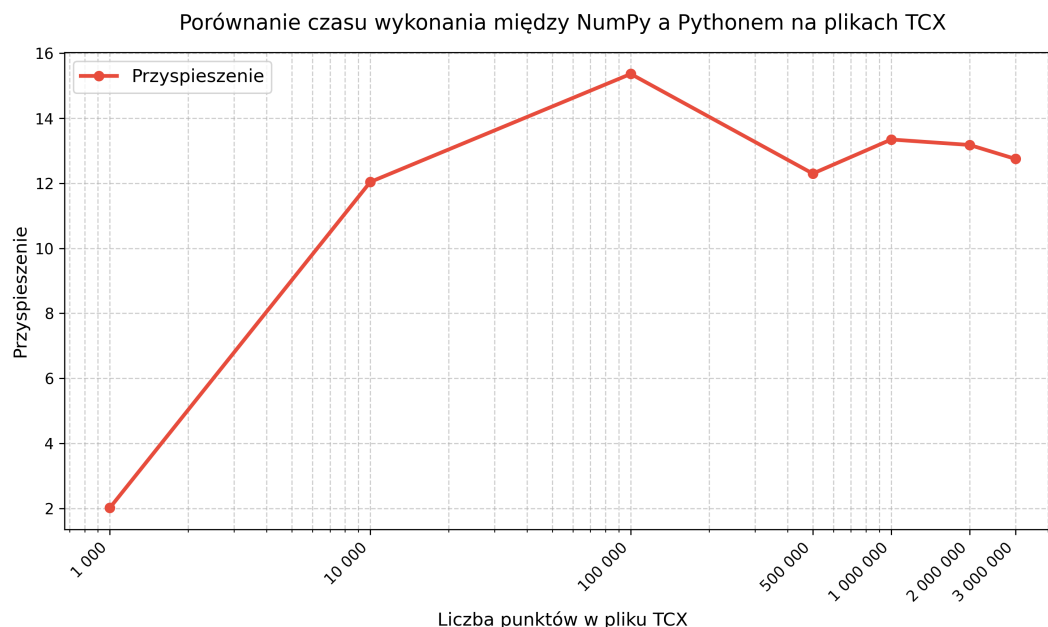
Liczba punktów	Czas – Python [s]	Czas – NumPy [s]	Przyspieszenie
1 000	0,0047	0,0023	2,01×
10 000	0,0445	0,0037	12,04×
100 000	0,7571	0,0493	15,36×
500 000	3,0563	0,2485	12,30×
1 000 000	6,2827	0,4707	13,35×
2 000 000	14,4916	1,1000	13,18×
3 000 000	22,6585	1,7773	12,75×

4.3.1 Czas przetwarzania

Analizując bezwzględne czasy wykonania, zaprezentowane na rysunku 4.1, można zaobserwować, że w przypadku obu technologii czas przetwarzania rośnie w przybliżeniu liniowo wraz ze wzrostem liczby punktów $\mathcal{O}(N)$. Różnica polega jednak na skali tego wzrostu.

Dla najmniejszego pliku (1 000 punktów) czasy wykonania obu implementacji są na tyle małe (rzędu pojedynczych milisekund), że różnica jest z perspektywy użytkownika niedostrzegalna. Różnice następują jednak przy większych zbiorach danych. Dla pliku zawierającego milion punktów TCX, klasyczne podejście oparte na standardowych listach i pętlach języka Python wymagało ponad 6 sekund na zakończenie obliczeń. Zastosowanie operacji wektorowych z wykorzystaniem biblioteki NumPy skróciło ten czas do

zaledwie około 0,47 sekundy. Wizualizacja wyraźnie ukazuje, że nachylenie krzywej wzrostu dla Pythona jest wielokrotnie bardziej strome, co czyni to podejście nieoptymalnym dla systemów przetwarzających duże ilości danych.



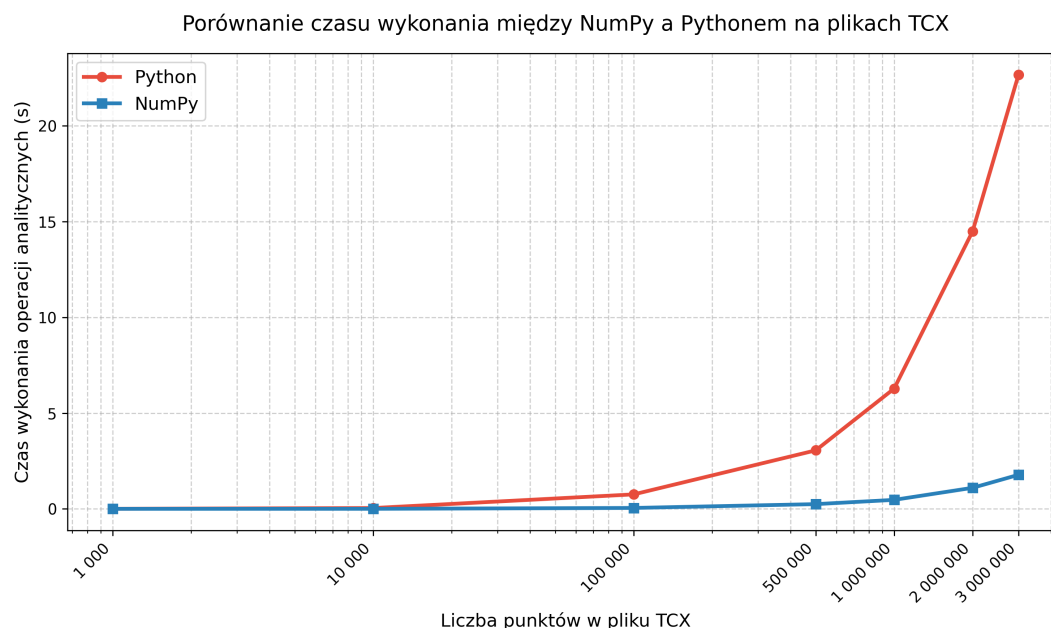
Rysunek 4.1: Porównanie czasu wykonania operacji analitycznych na plikach TCX

4.3.2 Analiza współczynnika przyspieszenia

Aby dokładniej zbadać dynamikę wydajności, obliczono współczynnik przyspieszenia (stosunek czasu wykonania na standardowych strukturach Pythona do czasu z wykorzystaniem NumPy), który zobrazowano na rysunku 4.2. Analiza tego wykresu dostarcza interesujące wnioski dotyczące narzutu technologicznego oraz architektury sprzętowej.

Dla bardzo małych zbiorów (1 000 punktów) przyspieszenie wynosi stosunkowo niewiele, bo zaledwie $2,01\times$. Zjawisko to może wynikać z faktu, że dla małych tablic czas samej alokacji obiektów NumPy i narzut związany z wywołaniem funkcji zewnętrznych w C dominuje nad właściwym czasem obliczeń wektorowych.

Gwałtowny skok wydajności i szczytowe przyspieszenie (blisko $15,4\times$) odnotowano dla pliku z 100 000 punktami. Na tym etapie narzut związany z inicjalizacją staje się marginalny. Wysokie przyspieszenie wynika prawdopodobnie z efektywnej wektoryzacji operacji oraz lepszego wykorzystania zasobów procesora. Przy dalszym zwiększaniu wolumenu danych (powyżej 100 000 punktów), przyspieszenie nieznacznie spada i stabilizuje się na poziomie $12\text{--}13\times$. Ten delikatny spadek z wartości szczytowej sugeruje, że przy bardzo dużych wektorach głównym wąskim gardłem przestaje być sama jednostka obliczeniowa, a staje się nim przepustowość magistrali pamięci i efektywność pamięci podręcznej.



Rysunek 4.2: Dynamika przyspieszenia (NumPy vs Python)

4.3.3 Szczegółowa analiza profilu operacji analitycznych

Aby zlokalizować źródła zysków w wydajności oferowanej przez bibliotekę NumPy, przeprowadzono profilowanie pojedynczych funkcji wchodzących w skład zestawu analitycznego. Badanie zrealizowano na największym zbiorze danych (1 000 000 punktów), wyznaczając średni czas wykonania każdej operacji na podstawie 50 niezależnych iteracji pomiarowych. Podejście to pozwoliło na wyeliminowanie losowych błędów pomiarowych. Wyniki tej analizy zebrano w tabeli 4.3.

Główny ciężar obliczeniowy w obu implementacjach spoczywa na dwóch najbardziej złożonych operacjach: `perf_eff` (wyznaczanie efektywności treningu) oraz `activity_segments` (segmentacja aktywności). W klasycznym podejściu pythonowym wykonanie samej funkcji `perf_eff` zajmuje aż 3,25 sekundy, co stanowi blisko 46% czasu całego benchmarku. Tak wysoki koszt czasowy wynika z konieczności sekwencyjnego iterowania po wielu natywnych listach (współrzędne, wysokość, tętno, czas) oraz obciążenia obliczaniem złożonych formuł fizycznych. Dodatkowym, niezwykle kosztownym krokiem jest algorytm wyznaczania znormalizowanej mocy, który w czystym Pythonie wymusza nieustanne, dynamiczne alokowanie nowych podlist w pamięci na potrzeby wyliczania 30-sekundowej średniej kroczącej. NumPy redukuje ten czas do zaledwie 0,20 sekundy, osiągając 16-krotne przyspieszenie poprzez eliminację narzutu interpretera na sprawdzanie typów „w locie”.

Najwyższy, ponad dwudziestokrotny wzrost wydajności odnotowano w przypadku operacji `hr_zones` (21,42×) oraz `avg_hr` (18,78×). Zjawisko to można łatwo uzasadnić

strukturą danych i sposobem ich ułożenia w pamięci operacyjnej. Obie te funkcje operują na pojedynczym, ciągłym wektorze danych (wartości tętne). Dla NumPy oznacza to idealne warunki do wykorzystania wektoryzacji, gdzie procesor w jednym cyklu zegara przetwarza wiele sąsiadujących ze sobą wartości liczbowych.

Tabela 4.3: Profil wydajnościowy poszczególnych funkcji analitycznych dla pliku zawierającego 1 000 000 punktów (średnia z 50 iteracji)

Nazwa funkcji	Czas – Python [s]	Czas – NumPy [s]	Przyspieszenie
<code>total_distance</code>	1,1209	0,1111	10,09×
<code>elevation_gain</code>	0,1616	0,0116	13,93×
<code>avg_hr</code>	0,1406	0,0075	18,78×
<code>hr_zones</code>	0,1080	0,0050	21,42×
<code>elevation_hr</code>	0,2404	0,0277	8,68×
<code>perf_eff</code>	3,2472	0,2024	16,04×
<code>activity_segments</code>	2,1243	0,2307	9,21×

Z kolei najniższe współczynniki przyspieszenia zaobserwowano dla algorytmów `elevation_hr` (8,68×), `activity_segments` (9,21×) oraz `total_distance` (10,09×). Aby poznać dokładne przyczyny niskich wyników, wykorzystano narzędzie z biblioteki `line_profiler`, które wykonuje dokładny pomiaru czasu wykonania dla każdej pojedynczej instrukcji. Przeprowadzona analiza wykazała, że niższy zysk wydajnościowy w tych przypadkach ma ściśle podłoże algorytmiczne:

- `elevation_hr` osiąga mniejsze przyspieszenie z uwagi na znaczne obciążenie związane z filtrowaniem danych. Wyniki z narzędzia `line_profiler` wykazały, że blisko 65% czasu wykonania tej funkcji pochłaniają zaledwie dwie linie kodu, które odpowiadają za wyodrębnienie tętna dla podbiegów i zbiegów. Wylizanie masek logicznych, a następnie kopiowanie pasujących elementów do nowych tablic w pamięci operacyjnej wymaga dużej alokacji zasobów, co wpływa na ograniczenie korzyści związanych z przetwarzania wektorowego.
- `total_distance` wyznacza odległość za pomocą funkcji `distance`, która z kolei korzysta ze złożonej formuły Haversine’a. Profilowanie ujawniło, że ponad 75% czasu w tej funkcji przypada na obliczanie operacji trygonometrycznych na (`np.sin`, `np.cos`, `np.arctan2`). Wektoryzacja tak rozbudowanego równania przy użyciu biblioteki NumPy wymusza wykonywanie każdego kroku na całych wektorach. Skutkuje to nieustannym alokowaniem ogromnych, tymczasowych tablic pośrednich

w pamięci. Te operacje mogą negatywnie wpływać na wydajność obliczeniową procesora.

- `activity_segments` to funkcja, w której specyfika problemu znacząco ogranicza możliwości optymalizacyjne biblioteki NumPy. Logika algorytmu oparta na śledzeniu maszyny stanów, filtrowaniu krótkich przerw oraz scalaniu sąsiadujących bloków wymusza analizę sekwencyjną. W rezultacie obie implementacje opierają swój główny rdzeń obliczeniowy na klasycznych pętlach `for`, a znaczna część kodu pozostaje w obu wariantach niemal identyczna. Dane z profilera udowodniły, że początkowe instrukcje wektorowe zajmują zaledwie kilkanaście procent czasu. Reszta narzutu, która stanowi około 80% całkowitego czasu wykonania funkcji jest generowana przez konieczność utrzymania pętli, które odpowiadają za sekwencyjne transformacje słowników.

Badanie poszczególnych składowych potwierdza, że NumPy osiąga najwyższą efektywność w zadaniach realizowanych na ciągłych strukturach jednowymiarowych, podczas gdy operacje wymagające zaawansowanego filtrowania i operacji logicznych na wielu powiązanych wektorach wykazują mniejszą przewagę nad standardową implementacją języka Python.

4.4 Analiza wykorzystania pamięci podręcznej

W celu zrozumienia różnic wydajnościowych pomiędzy podejściem na listach w języku Python a wektoryzacją za pomocą biblioteki NumPy, zdecydowano się na rozszerzenie badań o analizę chybień pamięci podręcznej procesora. Pozwala to wykazać, w jakim stopniu ciągłość danych w pamięci oraz lepsze zarządzanie jej strukturami przez bibliotekę NumPy wpływają na optymalizację czasu wykonania.

4.4.1 Metodyka i problematyka profilowania sprzętowego

Jak wspomniano w rozdziale poświęconym specyfikacji sprzętowej, głównym systemem operacyjnym wykorzystywanym w badaniach był Microsoft Windows 11. Choć system ten ma wbudowane narzędzia diagnostyczne, a producenci procesorów udostępniają dedykowane oprogramowanie profilujące (takie jak AMD uProf czy Intel VTune), bezpośredni pomiar chybień pamięci podręcznej dla procesów interpretera Pythona okazał się być problematyczny.

Wymusiło to poszukiwanie rozwiązań opartych na systemach z rodziny Linux, które dysponują narzędziami profilującymi, takimi jak `perf`. Aby uniknąć instalacji dodatkowego systemu operacyjnego, zdecydowano się na wykorzystanie podsystemu WSL 2 (ang. *Windows Subsystem for Linux*).

Uruchomienie profilowania w środowisku wirtualnym wymagało pokonania barier architektonicznych. W pierwszej kolejności konieczne było włączenie odpowiednich funkcji w systemie Windows, w tym przede wszystkim `Virtual Machine Platform` oraz `Windows Subsystem for Linux`.

Kolejnym etapem, z którym wiązały się trudności techniczne, była instalacja i uruchomienie samego narzędzia `perf` w dystrybucji Ubuntu pod kontrolą WSL. Ze względu na fakt, że WSL wykorzystuje dedykowane jądro dostarczane przez firmę Microsoft, instalacja standardowych pakietów skompilowanych dla czystego jądra Linuxa prawdopodobnie może prowadzić do rozbieżności wersji. W rezultacie standardowe wywołanie polecenia `perf` w terminalu kończyło się błędem braku kompatybilności. Instalacja pakietów generycznych (`linux-tools-generic`) pozwoliło obejść ten problem. Aby pomyślnie uruchomić profiler i zebrać dane, konieczne było każdorazowe odwoływanie się do narzędzia poprzez podanie pełnej ścieżki do odpowiedniego pliku, który znajduje się wewnątrz katalogu `/usr/lib/linux-tools/`.

Kolejnym wyzwaniem, wynikającym z wykorzystywanej technologii, była specyfika profilowania języków interpretowanych. Należy podkreślić, że `perf` mierzy pracę sprzętu w trakcie wykonywania programu, a nie samą logikę zapisaną w kodzie źródłowym. Oznacza to, że z perspektywy profilera badaniu nie podlegał bezpośrednio skrypt, lecz główny proces interpretera. Przekłada się to na fakt, że zliczane chybień pamięci podręcznej obejmowały również wewnętrzne mechanizmy maszyny wirtualnej Pythona.

Z tego powodu kluczowym problemem stało się odizolowanie metryk sprzętowych należących tylko i wyłącznie do badanych funkcji analitycznych od etapu parsowania plików TCX. Parsowanie danych w formacie XML jest z operacją generującą duży ruch w pamięci i ogromną liczbę alokacji, co w przypadku jednoczesnego pomiaru zaburzyłoby statystyki algorytmów analitycznych.

Aby rozwiązać ten problem i uzyskać wiarygodne dane procedura badawcza składała się z dwóch etapów:

1. W pierwszym kroku narzędzie `perf` monitorowało wykonanie skryptu ograniczającego się wyłącznie do wczytania i sparsowania danych z pliku TCX do pamięci. Pomiar z tego etapu stanowiły poziom odniesienia.
2. Następnie przeprowadzono drugi pomiar dla skryptu obejmującego parsowanie połączone z wywoływaniem pełnego zestawu funkcji analitycznych.

Obliczenie różnicy liczby chybień w pamięci podręcznej pomiędzy drugim a pierwszym przebiegiem pozwoliło na wyizolowanie narzutu generowanego wyłącznie przez implementację natywną Pythona oraz wariant oparty na bibliotece NumPy, z całkowitym wykluczeniem fazy wejścia i parsowania.

4.4.2 Analiza chybień pamięci podręcznej

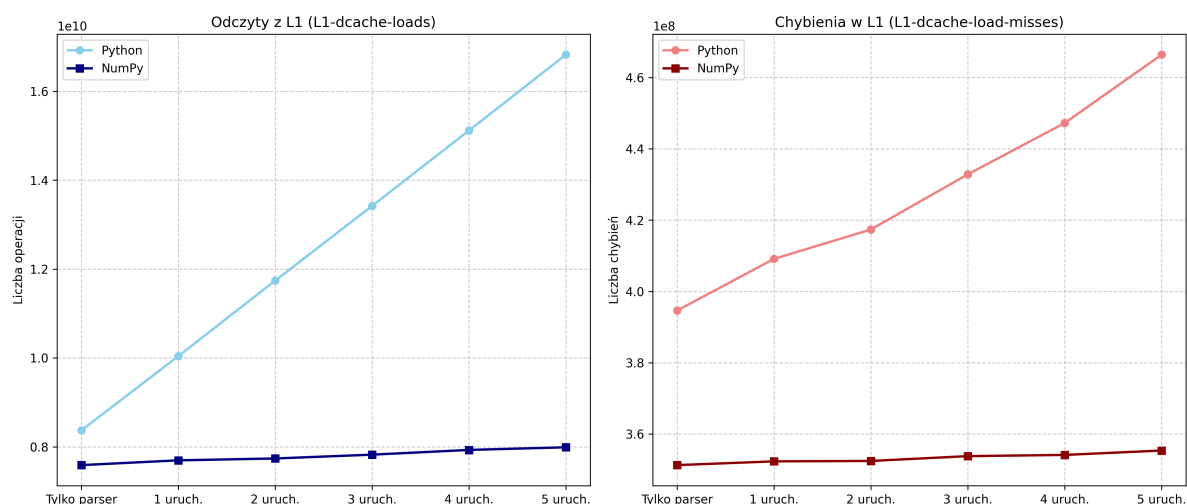
Zgodnie z przyjętą metodyką, pomiary sprzętowe za pomocą narzędzia `perf` przeprowadzono w czterech wariantach dla obu technologii jako pomiar bazowy samego parsera oraz jako serię jednego, dwóch i trzech następujących po sobie wywołań pełnego zestawu funkcji analitycznych. Pomiary wykonano dla pliku TCX, zawierającego 100 000 punktów. Otrzymane wyniki, które stanowiły średnią pięciu iteracji dla każdego wariantu, zostały zestawione w tabeli 4.4.

Podczas analizowania wyników zauważono, że ogólny wskaźnik chybień pamięci podręcznej (`cache-misses`) nie dawał jednoznacznych wzorców i trendów. Najbardziej prawdopodobną przyczyną były zakłócenia wynikające ze specyfiki środowiska wirtualnego oraz procesów działające w tle związanych z głównym systemem operacyjnym. Z tego względu w dalszej analizie skupiono się na zachowaniu pamięci podręcznej pierwszego poziomu dla danych (`L1-dcache`), ponieważ wykazywała względnie najbardziej regularny przebieg spośród analizowanych wskaźników, co zilustrowano na rysunku 4.3.

Tabela 4.4: Zestawienie zdarzeń sprzętowych dla pamięci podręcznej L1 (wartości uśrednione z 5 prób)

Wariant testowy	Odczyty z L1	Chyby w L1
Python: Tylko parser	8 369 252 311	394 634 661
Python: 1 uruchomienie	10 038 953 613	409 113 827
Python: 2 uruchomienia	11 738 099 075	417 363 342
Python: 3 uruchomienia	13 420 659 191	432 832 026
Python: 4 uruchomienia	15 118 810 439	447 218 087
Python: 5 uruchomień	16 824 287 410	466 380 500
NumPy: Tylko parser	7 587 314 363	351 291 033
NumPy: 1 uruchomienie	7 694 656 290	352 344 686
NumPy: 2 uruchomienia	7 736 910 257	352 460 586
NumPy: 3 uruchomienia	7 822 667 660	353 805 476
NumPy: 4 uruchomienia	7 929 017 424	354 150 054
NumPy: 5 uruchomień	7 988 174 091	355 380 221

Pierwszym i najbardziej rzucającym się w oczy wnioskiem jest dominacja fazy parsowania dokumentu TCX w ogólnym bilansie obciążenia procesora. W obu wariantach



Rysunek 4.3: Porównanie liczby odczytów oraz chybień dla pamięci podręcznej L1 w zależności od wariantu wykonania i zastosowanej technologii

technologicznych sam proces wczytywania pliku i budowy struktur pamięciowych wygenerował od 7,5 do ponad 8,3 miliarda żądań odczytu z pamięci podręcznej L1.

Ten wynik potwierdza słuszność zastosowania tej metodyki. Gdyby pomiar obejmował wykonanie całego skryptu, łącznie z fazą wejścia, znaczne obciążenie pamięci wynikające z parsowania struktury XML zdominowałoby ogólny bilans, maskując rzeczywisty koszt sprzętowy samej fazy analitycznej.

Wskaźnik, który może pomóc pokazać wyższość wektoryzacji nad klasycznymi strukturami języka Python jest różnica liczby żądań odczytu z pamięci danych L1 przy każdym kolejnym uruchomieniu obliczeń. Obserwując lewy wykres na rysunku 4.3, można zauważyć drastyczną różnicę w kącie nachylenia obu krzywych.

Średnio jedno pełne przejście algorytmów opartych na listach Pythona wymaga od procesora wykonania około 1,69 miliarda instrukcji pobrania danych z pamięci podręcznej. Z kolei dla implementacji wektorowej w bibliotece NumPy zjawisko to przyjmuje zupełnie mniejszą, generując średnio zaledwie około 80 milionów (0,08 mld) żądań na jedną iterację. Oznacza to, że kod oparty na wbudowanych strukturach Pythona angażuje kontroler pamięci podręcznej procesora blisko 21-krotnie częściej niż odpowiednik w NumPy dla dokładnie tego samego wolumenu danych.

Drugim aspektem obrazowanym na prawym wykresie (rysunek 4.3) jest wskaźnik **L1-dcache-load-misses**. Wskazuje on sytuacje, w których procesor zażądał danych, ale nie znalazł ich w najszybszej warstwie L1, co wymusiło opóźnienie i przeszukiwanie wolniejszych warstw pamięci.

Zestawienie danych ukazuje na wstępie ogromną pulę chybień wygenerowanych przez sam parser – ok. 395 mln dla klasycznej implementacji w Pythonie oraz 351 mln dla podejścia z NumPy. W klasycznym podejściu opartym na listach języka Python, poje-

Tabela 4.5: Różnice w liczbach odczytów i chybień pamięci L1 dla poszczególnych iteracji algorytmów

Różnice	Odczyty z L1		Chybień w L1	
	Python	NumPy	Python	NumPy
Parser / Iteracja 1	1 669 701 302	107 341 927	14 479 166	1 053 653
Iteracja 2 / Iteracja 1	1 699 145 462	42 253 967	8 249 515	115 900
Iteracja 3 / Iteracja 2	1 682 560 116	85 757 403	15 468 684	1 344 890
Iteracja 4 / Iteracja 3	1 698 151 248	106 349 764	14 386 061	344 578
Iteracja 5 / Iteracja 4	1 705 476 971	59 156 667	19 162 413	1 230 167
Średnia	1 691 007 019	80 171 945	14 349 167	817 837

dyncze wykonanie pełnego zestawu funkcji skutkuje średnio generowaniem 14,35 miliona dodatkowych chybień pamięci L1. Zoptymalizowany wariant z biblioteką NumPy obniża ten wynik do zaledwie około 0,82 miliona chybień na iterację.

Te wyniki dowodzą, że implementacja oparta na listach Pythona nie trafia w najszybszą pamięć podręczną procesora ponad 17,5 raza częściej niż ich wektorowe odpowiedniki. Zjawisko to wynika bezpośrednio z różnicy w lokalności przestrzennej. Wskaźnikowa budowa list Pythona i wynikająca z niej fragmentacja danych wymuszają kosztowne odpytywanie wolniejszych warstw pamięci operacyjnej. Z kolei alokacja wektorów NumPy w jednym, ciągłym bloku optymalizuje lokalność przestrzenną, co pozwala procesorowi na płynne, sekwencyjne ładowanie danych bezpośrednio do najszybszej pamięci L1.

4.4.3 Podsumowanie analizy wydajnościowej i sprzętowej

Przeprowadzone dwuetapowe badania, obejmujące zarówno pomiar czasu wykonania, jak i profilowanie zdarzeń sprzętowych, dostarczyły spójnych i jednoznacznych wniosków dotyczących przetwarzania dużych zbiorów danych telemetrycznych. Wyniki obu analiz udowadniają, że przewaga rozwiązań wektorowych biblioteki wynika z optymalnego wykorzystania fizycznej architektury procesorów oraz układów pamięci.

Z perspektywy czasu przetwarzania wykazano, że zastosowanie wektoryzacji pozwala na redukcję kosztu obliczeniowego do aż 13-krotnego przyspieszenia względem implementacji opartej na standardowych listach. Szczegółowe analiza pokazała, że największe zyski mogły sięgać nawet ponad 21-krotność przyspieszenia. Zidentyfikowano również ograniczenia wektoryzacji. Wykazano, że silnie sekwencyjna detekcja segmentów aktywności oraz brak mechanizmu fuzji pętli zmniejszały potencjał instrukcji SIMD.

Analiza chybień pamięci podręcznej dostarczyła uzasadnienia dla zaobserwowanych różnic czasowych. Pomiary z wykorzystaniem narzędzia `perf` udowodniły, że klasyczne podejście oparte na wbudowanych strukturach języka Python wymusza generowanie blisko 21-krotnie większej liczby żądań odczytu z pamięci pierwszego poziomu. Dodatkowo generuje ono niemal 17-krotnie więcej kosztownych chybień na każdą iterację algorytmu. Z kolei ułożenie danych w ciągłych blokach pamięci przez bibliotekę NumPy gwarantowało zachowanie wysokiej lokalności przestrzennej, umożliwiając optymalne działanie mechanizmów pobierania danych.

Podsumowanie

Celem pracy było zaprojektowanie i zaimplementowanie oprogramowania do analizy przestrzenno-czasowej danych treningowych pochodzących z urządzeń GPS. szczególnym uwzględnieniem przeprowadzenia analizy porównawczej wydajności różnych metod przetwarzania. Postawione cele zostały w pełni zrealizowane. Stworzono środowisko analityczne, na bazie którego zbadano i wykazano różnice między klasycznym modelem programowania w języku Python a wektorowym podejściem z wykorzystaniem biblioteki NumPy.

Podczas prac nad oprogramowaniem i w trakcie przeprowadzania testów udowodniono, jak duży wpływ na wydajność obliczeniową ma odpowiednia organizacja danych w pamięci komputera. Zastosowanie wektoryzacji pozwoliło na znaczną redukcję czasu obliczeń, wielokrotnie przyspieszając przetwarzanie dużych zbiorów danych. Profilowanie sprzętowe pokazało, że zoptymalizowane, ciągłe struktury pamięciowe pozwalają na znacznie lepsze zarządzanie zasobami procesora. W dobie rosnącej świadomości na temat zdrowego trybu życia oraz faktu, że współczesne urządzenia sportowe potrafią generować ogromne ilości danych, optymalizacja ich przetwarzania okazuje się bardzo istotna. Wyniki potwierdzają, że temat pracy jest aktualny i może mieć duże znaczenie praktyczne.

Projekt posiada również potencjał do dalszego rozwoju. Za pomocą rozszerzeń, format TCX może zostać wzbogacony o dodatkowe metryki, co daje szansę na bardziej rozbudowane przetwarzanie tych danych. W przyszłości można byłoby przeprowadzić bardziej zaawansowane analizy, które znacznie głębiej weszłyby w zgromadzone struktury danych i poddały je rygorystycznemu badaniu. Przyszłe wersje projektu mogłyby wdrożyć zaawansowane modele matematyczne do predykcji trendów zmęczenia, estymacji pułapu tlenowego czy analizy zmienności rytmu zatokowego. Głębsze badanie struktur przestrzennych pozwoliłoby z kolei na zastosowanie złożonych algorytmów filtracji i korekty danych GPS.

Bibliografia

- [1] Ulrich Drepper, *What Every Programmer Should Know About Memory*, Red Hat, Inc., 2007
- [2] Agner Fog, *Optimizing software in C++: An optimization guide for Windows, Linux, and Mac platforms*, Agner Fog, 2025
- [3] John L. Hennessy, J. L., David A. Patterson, *Computer Architecture: A Quantitative Approach Fifth Edition*, Morgan Kaufmann, 2017
- [4] Richard Acton, *Data-Oriented Design: Software engineering for limited resources and short schedules*, Richard Acton, 2018
- [5] Bernhard Hofmann-Wellenhof, Herbert Lichtenegger, Elmar Wasle, *GNSS – Global Navigation Satellite Systems: GPS, GLONASS, Galileo, and more*, SpringerWien-NewYork, Austria, 2008
- [6] TopoGrafix, *GPX: the GPS Exchange Format*, <https://www.topografix.com/gpx.asp>, [dostęp: 14.04.2026]
- [7] Garmin, *Flexible and Interoperable Data Transfer (FIT) SDK*, <https://developer.garmin.com/fit/overview/>
- [8] Garmin, *Training Center Database XML Schema*, <https://www8.garmin.com/xmlschemas/TrainingCenterDatabasev2.xsd>, [dostęp: 14.04.2026]
- [9] TOP500, *TOP500's The List*, <https://www.top500.org/lists/top500/2025/11/>, [dostęp: 06.05.2026]
- [10] ETH Zurich, *HPC Documentation*, <https://docs.hpc.ethz.ch/>, [dostęp: 06.05.2026]
- [11] Björn Andrist, Viktor Sehr, *C++ High Performance: Master the art of optimizing the performance of your C++ applications*, Packt Publishing, Birmingham, 2018
- [12] Jake VanderPlas, *Python Data Science Handbook*, O'Reilly Media, Inc., 2016

-
- [13] Python Software Foundation, *Python 3 Documentation*, <https://docs.python.org/3/>, [dostęp: 06.05.2026]
 - [14] Kevin Williams, Jason McCandless, David Gregg, *Dynamic Interpretation for Dynamic Scripting Languages*, Department of Computer Science, Trinity College Dublin, Dublin, 2009
 - [15] NumPy Developers, *NumPy Documentation* <https://numpy.org/doc/2.4/>
 - [16] lxml Developers, *lxml - XML and HTML with Python*, <https://lxml.de/index.html>
 - [17] Matplotlib development team, *Matplotlib 3.10.9 documentation*, <https://matplotlib.org/stable/index.html>

Spis listingów

1.1	Przykładowy plik TCX	13
2.1	Porównanie operacji na listach Pythona a wektorach NumPy	23
3.1	Funkcja parsująca	26
3.2	Funkcja parsująca (NumPy)	27
3.3	Funkcje odpowiedzialne za obliczanie dystansu	29
3.4	Funkcje odpowiedzialne za obliczanie dystansu (NumPy)	30
3.5	Funkcje odpowiedzialne za obliczanie przewyższeń (listy, wektory)	31
3.6	Funkcje odpowiedzialne za analizę tętna	31
3.7	Funkcje odpowiedzialne za analizę tętna (NumPy)	33
3.8	Analiza wydolności biomechanicznej i tlenowej	34
3.9	Analiza wydolności biomechanicznej i tlenowej (NumPy)	36
3.10	Klasyfikacja stanów aktywności	38
3.11	Dzielenie segmentów aktywności (Python)	40
3.12	Formatowanie danych wyjściowych (Python)	41
3.13	Segmentacja aktywności podczas treningu (NumPy)	42
4.1	Generator syntetycznych danych treningowych w formacie TCX	46
4.2	Porównanie czasu wykonania algorytmów	49

Spis rysunków

4.1	Porównanie czasu wykonania operacji analitycznych na plikach TCX . . .	52
4.2	Dynamika przyspieszenia (NumPy vs Python)	53
4.3	Porównanie liczby odczytów oraz chybień dla pamięci podręcznej L1 w zależności od wariantu wykonania i zastosowanej technologii	58

Spis tabel

4.1	Specyfikacja środowiska sprzętowo-programowego	46
4.2	Zestawienie czasów wykonania algorytmów analitycznych (w sekundach)	51
4.3	Profil wydajnościowy poszczególnych funkcji analitycznych dla pliku zawierającego 1 000 000 punktów (średnia z 50 iteracji)	54
4.4	Zestawienie zdarzeń sprzętowych dla pamięci podręcznej L1 (wartości uśrednione z 5 prób)	57
4.5	Różnice w liczbach odczytów i chybień pamięci L1 dla poszczególnych iteracji algorytmów	59